

TechTribers

Java Interview Series · Part 18 · Q850–Q924

Stream API Coding Questions

75 interview problems solved idiomatically: filtering and searching, map and flatMap, sorting and ranking, every major collector, the employee-dataset classics, string and character puzzles, and modern features from takeWhile to Gatherers, each with its real output.

75	75	6	17–25
questions	code examples	diagrams	Java LTS

How to use this guide

Every question follows the same layout so you can revise fast the night before and go deep when you have time. Read the **Quick answer** first. If you can say it in your own words, move on. If not, read the explanation and run the code.

Q07 SA-007

Number and ID. Q-numbers run continuously across the whole TechTribers Java series, so every question has one unique number. The topic ID stays fixed across editions.

• Basic • Intermediate • Advanced

Difficulty. Basic for 0–2 years, Intermediate for 2–5 years, Advanced for senior rounds.

⚡ Quick answer

The one-line answer. Lead with this in the interview, then expand.

⚠️ **Interview trap**
A common wrong answer or an edge case interviewers use to filter candidates.

💡 **Pro tip**
What a senior engineer would add: production practice, design reasoning, modern Java.

🎯 **Likely follow-up**
The next question the interviewer is likely to ask, with the answer.

75	7	75	6
questions	sections	code examples	diagrams

Examples target modern Java (17, 21 and 25 LTS). Where a feature needs a newer version, the answer says which one.

© TechTribers. For personal use only. Do not redistribute.

Contents

🔎	01	Filtering, Matching and Searching	Q850–Q860	6
🔁	02	Mapping, flatMap and Transformation	Q861–Q871	14
📊	03	Sorting, Distinct, Limits and Ranking	Q872–Q882	23
🧺	04	Collectors: Grouping, Partitioning and Reducing	Q883–Q895	31
👥	05	Employee Dataset Scenarios	Q896–Q907	42
🏭	06	Real-World Data Pipelines	Q908–Q916	51
⚙️	07	Advanced: Generation, Laziness and Modern Stream Features	Q917–Q924	58

© TechTribers. For personal use only. Do not redistribute.

All questions

🔎 Filtering, Matching and Searching

Q850	Filter the even numbers from a list and return their squares.	6
Q851	Remove nulls and blank strings from a list, trimming what remains.	7
Q852	Count the strings that start with a given prefix, ignoring case.	7
Q853	Use `anyMatch`, `allMatch` and `noneMatch`, including their behaviour on an empty list.	8
Q854	Find the first element greater than a threshold; when do you use `findFirst` vs `findAny`?	9
Q855	Find the common elements (intersection) and the difference between two lists efficiently.	9
Q856	Filter a `Map` by value and collect the matching keys, keeping a predictable order.	10
Q857	Check whether a list is sorted in ascending order.	11
Q858	Compute latency percentiles (p50, p95, p99) from a list of measurements.	12
Q859	Use `takeWhile` and `dropWhile`, and explain how they differ from `filter`.	12
Q860	Find all indexes at which a value occurs in a list.	13

🔁 Mapping, flatMap and Transformation

Q861	Convert a list of words to uppercase, and build a word-to-length map.	14
Q862	Flatten a `List<List<Integer>>` and an `int[][]` into one sequence.	15
Q863	Generate the cartesian product of two lists, and find all pairs that add up to a target.	16
Q864	Split a list of sentences into distinct, lower-case words.	16
Q865	Convert a list of objects to a `Map`, handling duplicate keys with a merge rule.	17
Q866	Combine two lists into a map by position (zip keys with values).	18
Q867	Turn a map into a sorted list of `"key=value"` strings.	18
Q868	Convert between `int[]`, `Integer[]` and `List<Integer>` with streams.	19
Q869	Compute sliding windows of size k and a running (prefix) sum.	20
Q870	Pair each element with its index (zipWithIndex) and print a numbered list.	21
Q871	Expand each order into its line items with `mapMulti` (Java 16), and compare it with `flatMap`.	21

📊 Sorting, Distinct, Limits and Ranking

Q872	Sort numbers in descending order, and sort strings by length then alphabetically.	23
Q873	Get distinct elements by a property (distinct-by-key), keeping the first occurrence.	24
Q874	Find the duplicate elements in a list (and how many times each occurs).	25
Q875	Find the maximum and minimum, and the longest string, handling ties.	25
Q876	Find the Nth largest distinct number (for example, the 3rd largest).	26
Q877	Find the second highest number, and explain the duplicate trap.	27
Q878	Get the top N and bottom N elements by a score.	27
Q879	Implement pagination with `skip` and `limit` (page number and page size).	28
Q880	Sort a map by value (descending) and keep the result as an ordered map.	29
Q881	Sort elements by frequency (most frequent first), breaking ties by value.	29
Q882	Merge two lists, remove duplicates and sort; how does `Stream.concat` behave?	30

© TechTribers. For personal use only. Do not redistribute.

🧺 Collectors: Grouping, Partitioning and Reducing

Q883 Group words by length, with sorted keys and alphabetically sorted groups. 31

Q884 Choose the container for each group: `toSet`, `toCollection(TreeSet::new)` or `toUnmodifiableList`. 32

Q885 Partition numbers into even and odd, and count each side. 33

Q886 Group names by first letter and join each group into one comma-separated string. 33

Q887 Build a two-level grouping: department, then city, then the list of names. 34

Q888 Compute count, sum, min, max and average in one pass with summary statistics. 35

Q889 Sum values per key with `toMap` merging (aggregate order amounts by customer). 36

Q890 Use `collectingAndThen` to return an unmodifiable list and to unwrap `maxBy` results. 36

Q891 Compute two results in one pass with `Collectors.teeing` (Java 12). 37

Q892 Use the `filtering` and `flatMapping` downstream collectors (Java 9). 38

Q893 Use `reduce` for a product and a sum of squares, and know when not to use it. 39

Q894 Write a custom `Collector` that keeps the top K elements (min-heap) in one pass. 40

Q895 Get counts as `Integer` instead of `Long`, and group into an `EnumMap`. 41

👥 Employee Dataset Scenarios

Q896 Group employees by department and list their names, sorted. 42

Q897 Count employees per department, and find the department with the most employees. 43

Q898 Compute the average salary per department, rounded, highest first. 43

Q899 Find the highest-paid employee in each department (with ties). 44

Q900 Find the second-highest salary and every employee who earns it. 45

Q901 Find employees who earn more than their department's average salary. 46

Q902 Compute the total salary bill per city and find the costliest city. 47

Q903 List employees who joined after 2018, sorted by joining year and then name. 47

Q904 Find the oldest and youngest employee, and the average age by gender. 48

Q905 Give a 10% raise to one department without mutating the original list. 48

Q906 Partition employees into those earning above and below 1,00,000, listing names. 49

Q907 Produce salary statistics per department in one pass. 50

🏭 Real-World Data Pipelines

Q908 Parse raw text lines into records, skipping the header and rejecting malformed rows. 51

Q909 Join orders with customers in memory, like a SQL inner join. 52

Q910 Find customers who have no orders (an anti-join) with streams. 53

Q911 Aggregate transactions by month (`YearMonth`) and compute monthly totals. 53

Q912 Build a pivot table (region by quarter) with zero-filled cells. 54

Q913 Deduplicate events by ID, keeping only the latest version of each. 55

Q914 Bucket values into ranges (a histogram of salary bands). 56

Q915 Reconcile two datasets by key: matched, changed, only in source, only in target. 56

Q916 Flatten a recursive tree of categories into a list with depth and path. 57

© TechTribers. For personal use only. Do not redistribute.

⚙️ Advanced: Generation, Laziness and Modern Stream Features

Q917	Generate the first N Fibonacci numbers and the first N primes with streams.	58
Q918	Create infinite streams safely with `iterate`, `generate` and seeded random numbers.	59
Q919	Compute a large factorial and other aggregates with primitive streams and `BigInteger`.	60
Q920	What does this pipeline print? (lazy evaluation, operation order and short-circuiting)	61
Q921	Use Stream Gatherers (Java 24) for a running sum, fixed and sliding windows.	62
Q922	Why can a stream be consumed only once, and how do you reuse a pipeline?	63
Q923	Drop empty `Optional`s from a stream and flatten the present values.	64
Q924	List every Monday between two dates, and count working days, with `LocalDate.datesUntil`.	64

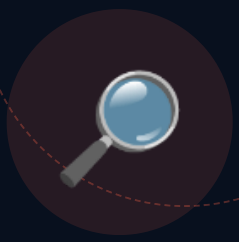
© TechTribers. For personal use only. Do not redistribute.

Section 01

Filtering, Matching and Searching

filter, the match operations and their empty-stream rules, findFirst versus findAny, set-based intersection and difference, filtering maps, sortedness checks, first unique element, takeWhile/dropWhile and index searches.

Questions 850–860 · 11 questions



Q850 SA-001

• Basic

Filter the even numbers from a list and return their squares.

⚡ Quick answer

Chain `filter` (keep elements matching a predicate) and `map` (transform each kept element), then collect. For numbers, `mapToInt` avoids boxing when you only need a sum or an `int[]`.

Java

```
List<Integer> numbers = List.of(3, 8, 5, 12, 7, 6, 10);

List<Integer> evenSquares = numbers.stream()
        .filter(n -> n % 2 == 0)
        .map(n -> n * n)
        .toList();

int sumOfEvenSquares = numbers.stream()
        .filter(n -> n % 2 == 0)
        .mapToInt(n -> n * n)
        .sum();

System.out.println(evenSquares);
System.out.println(sumOfEvenSquares);
```

▶ Output

```
[64, 144, 36, 100]
344
```

💡 Pro tip

`n % 2 == 0` is correct for negative numbers too; the common bug is `n % 2 == 1` for odd numbers, which fails for negatives (`-3 % 2 == -1`). Use `n % 2 != 0`.

© TechTribers. For personal use only. Do not redistribute.

Q851 SA-002

• Basic

Remove nulls and blank strings from a list, trimming what remains.

⚡ Quick answer

Filter with `Objects::nonNull` first, then `map(String::strip)` and drop empty results. Order matters: calling a method on an element before the null check throws `NullPointerException`.

Java

```
List<String> raw = Arrays.asList("  pune ", null, "", "   ", "mumbai", null, " delhi");

List<String> cleaned = raw.stream()
        .filter(Objects::nonNull)
        .map(String::strip)
        .filter(s -> !s.isEmpty())
        .toList();

System.out.println(cleaned);
```

▶ Output

```
[pune, mumbai, delhi]
```

⚠️ Interview trap

`List.of(...)` rejects `null` elements at creation time, which is why the input uses `Arrays.asList`. A stream from `List.of` can never contain nulls.

Q852 SA-003

• Basic

Count the strings that start with a given prefix, ignoring case.

⚡ Quick answer

`filter` plus `count()`. For a case-insensitive prefix test, `regionMatches(true, 0, prefix, 0, prefix.length())` avoids creating a lower-cased copy of every string.

Java

```
List<String> cities = List.of("Pune", "Patna", "Mumbai", "pondicherry", "Delhi", "PANAJI");
String prefix = "p";

long count = cities.stream()
        .filter(c -> c.regionMatches(true, 0, prefix, 0, prefix.length()))
        .count();

List<String> matches = cities.stream()
        .filter(c -> c.toLowerCase(Locale.ROOT).startsWith(prefix.toLowerCase(Locale.ROOT)))
        .toList();
```


© TechTribers. For personal use only. Do not redistribute.

```
System.out.println(count + " " + matches);
```

▶ Output

```
4 [Pune, Patna, pondicherry, PANAJI]
```

💡 Pro tip

Always pass `Locale.ROOT` to `toLowerCase` / `toUpperCase` for non-display logic; with a Turkish default locale, `"TITLE".toLowerCase()` contains a dotless ı.

✅ Q853 SA-004 • Intermediate

Use `anyMatch`, `allMatch` and `noneMatch`, including their behaviour on an empty list.

⚡ Quick answer

All three are **short-circuiting** terminal operations. On an **empty** stream, `anyMatch` returns `false`, while `allMatch` and `noneMatch` return `true` (vacuous truth). That edge case causes real bugs, such as "all orders shipped" being true for a customer with no orders.

Java

```
List<Integer> scores = List.of(72, 88, 95, 61);
List<Integer> none = List.of();

System.out.println("any >= 90: " + scores.stream().anyMatch(s -> s >= 90));
System.out.println("all >= 60: " + scores.stream().allMatch(s -> s >= 60));
System.out.println("none < 50: " + scores.stream().noneMatch(s -> s < 50));
System.out.println("empty -> any=" + none.stream().anyMatch(s -> s > 0)
        + " all=" + none.stream().allMatch(s -> s > 0)
        + " none=" + none.stream().noneMatch(s -> s > 0));

boolean allShipped = !none.isEmpty() && none.stream().allMatch(s -> s > 0);
System.out.println("guarded allMatch on empty: " + allShipped);
```

▶ Output

```
any >= 90: true
all >= 60: true
none < 50: true
empty -> any=false all=true none=true
guarded allMatch on empty: false
```

⚠️ Interview trap

Business rules like "every line item is approved" usually need `!list.isEmpty() && list.stream().allMatch(...)`. Decide explicitly what an empty input should mean.

© TechTribers. For personal use only. Do not redistribute.

Q854 SA-005

Intermediate

Find the first element greater than a threshold; when do you use `findFirst` vs `findAny`?

⚡ Quick answer

`filter(...).findFirst()` returns an `Optional` with the **first matching element in encounter order**. `findAny` may return **any** match and is only faster for **parallel** streams, where it doesn't have to respect order. On a sequential stream they behave the same in practice, but only `findFirst` guarantees it.

Java

```
List<Integer> latenciesMs = List.of(12, 48, 230, 19, 510, 75);

Optional<Integer> firstSlow = latenciesMs.stream().filter(ms -> ms > 200).findFirst();
System.out.println(firstSlow.map(ms -> "first slow call: " + ms + " ms").orElse("all fast"));

int firstOver1s = latenciesMs.stream().filter(ms -> ms > 1_000).findFirst().orElse(-1);
System.out.println("over 1 s: " + firstOver1s);

OptionalInt firstIndex = IntStream.range(0, latenciesMs.size())
        .filter(i -> latenciesMs.get(i) > 200)
        .findFirst();
System.out.println("index of first slow call: " + firstIndex.getAsInt());
```

▶ Output

```
first slow call: 230 ms
over 1 s: -1
index of first slow call: 2
```

💡 Pro tip

Prefer `orElseThrow()` (Java 10) over `get()`: same behaviour, but the name makes the failure path visible in review.

Q855 SA-006

Intermediate

Find the common elements (intersection) and the difference between two lists efficiently.

⚡ Quick answer

Convert the lookup side to a `HashSet` first, then `filter(set::contains)` for the intersection and `filter(not(set::contains))` for the difference. Filtering with `list2::contains` directly is **O(n×m)**, which turns into seconds once both lists have tens of thousands of elements.

The result keeps the **order of the streamed list**, because only the lookup side is a set. If order doesn't matter, `Set.retainAll` / `removeAll` on a copy is an equally good non-stream answer.

`Predicate.not` (Java 11) reads better than `s -> !todaySet.contains(s)` and works with method references.

© TechTribers. For personal use only. Do not redistribute.

```java
List<String> yesterday = List.of("asha", "ravi", "meera", "karan", "neha");
List<String> today = List.of("ravi", "neha", "vikram", "asha", "divya");

Set<String> todaySet = new HashSet<>(today);
Set<String> yesterdaySet = new HashSet<>(yesterday);

List<String> stillActive = yesterday.stream().filter(todaySet::contains).toList();
List<String> churned = yesterday.stream().filter(Predicate.not(todaySet::contains)).toList();
List<String> newUsers = today.stream().filter(Predicate.not(yesterdaySet::contains)).toList();

System.out.println("common:  " + stillActive);
System.out.println("churned: " + churned);
System.out.println("new:     " + newUsers);
```

▶ Output

```
common:  [asha, ravi, neha]
churned: [meera, karan]
new:     [vikram, divya]
```

Q856 SA-007 • Intermediate

Filter a `Map` by value and collect the matching keys, keeping a predictable order.

⚡ Quick answer

Stream `map.entrySet()`, filter on `getValue()`, then map to `getKey()` or collect back to a map. Use a `LinkedHashMap` (or `TreeMap`) supplier in `toMap` when the order of the result matters; a plain `toMap` returns a `HashMap` with no defined order.

```java
Map<String, Integer> stock = new LinkedHashMap<>();
stock.put("SKU-1", 0); stock.put("SKU-2", 14); stock.put("SKU-3", 3);
stock.put("SKU-4", 0); stock.put("SKU-5", 42);

List<String> outOfStock = stock.entrySet().stream()
        .filter(e -> e.getValue() == 0)
        .map(Map.Entry::getKey)
        .toList();

Map<String, Integer> lowStock = stock.entrySet().stream()
        .filter(e -> e.getValue() > 0 && e.getValue() < 10)
        .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue,
                (a, b) -> a, LinkedHashMap::new));

System.out.println(outOfStock + " " + lowStock);
```


© TechTribers. For personal use only. Do not redistribute.

▶ Output

```
[SKU-1, SKU-4] {SKU-3=3}
```

💡 Pro tip

For a simple in-place removal, `stock.values().removeIf(v -> v == 0)` or `stock.entrySet().removeIf(...)` is shorter than building a new map.

Q857 SA-008 • Intermediate

Check whether a list is sorted in ascending order.

⚡ Quick answer

Compare each element with its neighbour: `IntStream.range(0, size - 1).allMatch(i -> list.get(i) <= list.get(i + 1))`. It short-circuits at the first out-of-order pair. For a generic version, take a `Comparator<? super T>`.

Java

```
static <T> boolean isSorted(List<T> list, Comparator<? super T> cmp) {
    return IntStream.range(0, list.size() - 1)
            .allMatch(i -> cmp.compare(list.get(i), list.get(i + 1)) <= 0);
}

System.out.println(isSorted(List.of(1, 3, 3, 8, 12), Comparator.naturalOrder()));
System.out.println(isSorted(List.of(1, 9, 4), Comparator.naturalOrder()));
System.out.println(isSorted(List.of("kiwi", "fig", "apple"), Comparator.reverseOrder()));
System.out.println(isSorted(List.<Integer>of(), Comparator.naturalOrder()));
```

▶ Output

```
true
false
true
true
```

⚠️ Interview trap

Only valid for `RandomAccess` lists. On a `LinkedList`, `get(i)` is O(n) and the check becomes O(n²); iterate with an `Iterator` instead.

© TechTribers. For personal use only. Do not redistribute.

Q858 SA-009

• Intermediate

Compute latency percentiles (p50, p95, p99) from a list of measurements.

⚡ Quick answer

Sort once into a primitive array, then read percentiles by index using the **nearest-rank** method: index = `ceil(p / 100 × n) - 1`. Streams make the sort and conversion one line; do the percentile lookup on the array, not with repeated `skip`, so each percentile is O(1) after the sort.

The mean (59.95 ms here) is dragged up by two outliers, while p50 shows the typical request. That's why service-level objectives are written in percentiles. For production metrics, use a streaming estimator (HdrHistogram, Micrometer's percentile histograms); sorting every measurement only works for bounded samples.

Java

```java
static int percentile(int[] sorted, double p) {
    int rank = (int) Math.ceil(p / 100.0 * sorted.length);
    return sorted[Math.max(0, rank - 1)];
}

List<Integer> latenciesMs = List.of(12, 15, 11, 240, 18, 14, 16, 13, 95, 17,
        12, 19, 14, 16, 610, 15, 13, 18, 14, 17);

int[] sorted = latenciesMs.stream().mapToInt(Integer::intValue).sorted().toArray();

for (double p : new double[] {50, 90, 95, 99}) {
    System.out.printf("p%.0f = %d ms%n", p, percentile(sorted, p));
}
double mean = latenciesMs.stream().mapToInt(Integer::intValue).average().orElse(0);
System.out.println("mean = " + mean);
```

▶ Output

```
p50 = 15 ms
p90 = 95 ms
p95 = 240 ms
p99 = 610 ms
mean = 59.95
```

Q859 SA-010

• Intermediate

Use `takeWhile` and `dropWhile`, and explain how they differ from `filter`.

⚡ Quick answer

`takeWhile` (Java 9) emits elements **while** the predicate holds and **stops at the first failure**; `dropWhile` skips elements while it holds and emits **everything after**, including later elements that would fail. `filter` tests **every** element. On sorted data `takeWhile` is a short-circuiting `filter`; on unsorted data the results differ.

© TechTribers. For personal use only. Do not redistribute.

```java
List<Integer> sortedPrices = List.of(99, 149, 199, 249, 499, 999);
List<Integer> unsorted = List.of(99, 149, 999, 199, 249);

System.out.println("takeWhile < 300: " + sortedPrices.stream().takeWhile(p -> p < 300).toList());
System.out.println("dropWhile < 300: " + sortedPrices.stream().dropWhile(p -> p < 300).toList());
System.out.println("unsorted takeWhile: " + unsorted.stream().takeWhile(p -> p < 300).toList());
System.out.println("unsorted filter:    " + unsorted.stream().filter(p -> p < 300).toList());

List<String> log = List.of("# header", "# generated", "id,name", "1,asha", "# trailing");
System.out.println(log.stream().dropWhile(l -> l.startsWith("#")).toList());
```

▶ Output

```
takeWhile < 300: [99, 149, 199, 249]
dropWhile < 300: [499, 999]
unsorted takeWhile: [99, 149]
unsorted filter:    [99, 149, 199, 249]
[id,name, 1,asha, # trailing]
```

💡 Pro tip

`takeWhile` works on infinite streams: `Stream.iterate(1, x -> x * 2).takeWhile(x -> x < 1000)` terminates, while `filter` on an infinite stream never would.

📍 Q860 SA-011 • Basic

Find all indexes at which a value occurs in a list.

⚡ Quick answer

Stream the **indexes** with `IntStream.range(0, list.size())` and filter on the element at each index. The same pattern solves "position of max", "pairs of adjacent elements" and anything else that needs positions.

```java
List<String> events = List.of("LOGIN", "VIEW", "ERROR", "VIEW", "ERROR", "LOGOUT");

List<Integer> errorAt = IntStream.range(0, events.size())
        .filter(i -> events.get(i).equals("ERROR"))
        .boxed()
        .toList();

int indexOfMaxLen = IntStream.range(0, events.size())
        .reduce((a, b) -> events.get(b).length() > events.get(a).length() ? b : a)
        .orElse(-1);

System.out.println(errorAt + " " + indexOfMaxLen);
```


© TechTribers. For personal use only. Do not redistribute.

▶ Output

```
[2, 4] 5
```

Section 02

Mapping, flatMap and Transformation

map and method references, flattening nested lists and arrays, cartesian products and pair sums, splitting text into words, list-to-map conversions with duplicate keys, zipping two lists, boxing conversions, sliding windows, index pairing and mapMulti.

Questions 861–871 · 11 questions



Q861 SA-012 • Basic

Convert a list of words to uppercase, and build a word-to-length map.

⚡ Quick answer

`map(String::toUpperCase)` transforms each element. `Collectors.toMap(Function.identity(), String::length)` builds a map, but throws `IllegalStateException` on duplicate keys and gives no order guarantee, so pass a merge function and a map supplier when either matters.

Java

```java
List<String> words = List.of("stream", "lambda", "record", "stream", "sealed");

List<String> upper = words.stream().map(w -> w.toUpperCase(Locale.ROOT)).toList();

Map<String, Integer> lengths = words.stream()
        .collect(Collectors.toMap(Function.identity(), String::length,
                (first, dup) -> first, TreeMap::new));

System.out.println(upper);
System.out.println(lengths);
```

▶ Output

```
[STREAM, LAMBDA, RECORD, STREAM, SEALED]
{lambda=6, record=6, sealed=6, stream=6}
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**
Without the merge function, the duplicate `stream` throws `IllegalStateException: Duplicate key stream`.

Q862 SA-013 • Basic

Flatten a `List<List<Integer>>` and an `int[][]` into one sequence.

⚡ **Quick answer**
`flatMap(List::stream)` replaces each inner list with its elements. For primitive arrays use `flatMapToInt(Arrays::stream)` to stay in an `IntStream` and avoid boxing.

Java

```
List<List<Integer>> batches = List.of(List.of(1, 2, 3), List.of(), List.of(4, 5), List.of(6));
int[][] grid = { {1, 2}, {3, 4, 5}, {6} };

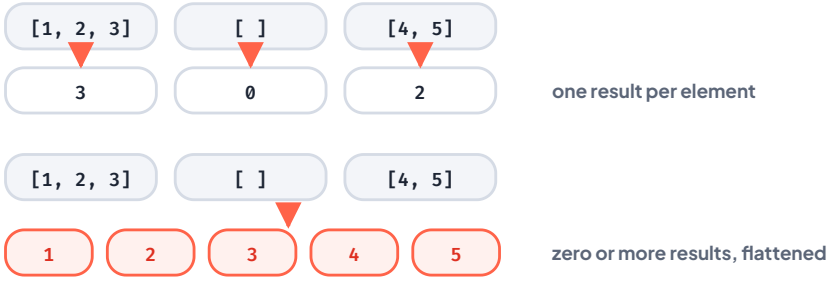
List<Integer> flat = batches.stream().flatMap(List::stream).toList();
int[] cells = Arrays.stream(grid).flatMapToInt(Arrays::stream).toArray();
int total = Arrays.stream(grid).flatMapToInt(Arrays::stream).sum();

System.out.println(flat);
System.out.println(Arrays.toString(cells) + " sum=" + total);
```

▶ **Output**

```
[1, 2, 3, 4, 5, 6]
[1, 2, 3, 4, 5, 6] sum=21
```

map vs flatMap



© TechTribers. For personal use only. Do not redistribute.

Q863 SA-014

• Intermediate

Generate the cartesian product of two lists, and find all pairs that add up to a target.

⚡ Quick answer

Nest a stream inside `flatMap` : for each element of the first list, map every element of the second to a pair. For pair sums, iterate **indexes** with `j > i` so each pair is reported once and an element isn't paired with itself. That's O(n²); for large inputs use a hash set in one pass.

Java

```
record Pair(int a, int b) { public String toString() { return "(" + a + "," + b + ")"; } }

List<String> sizes = List.of("S", "M");
List<String> colours = List.of("red", "blue", "black");
List<String> variants = sizes.stream()
        .flatMap(s -> colours.stream().map(c -> s + "-" + c))
        .toList();
System.out.println(variants);

List<Integer> nums = List.of(2, 7, 4, 5, 1, 6, 3);
int target = 8;
List<Pair> pairs = IntStream.range(0, nums.size()).boxed()
        .flatMap(i -> IntStream.range(i + 1, nums.size())
                .filter(j -> nums.get(i) + nums.get(j) == target)
                .mapToObj(j -> new Pair(nums.get(i), nums.get(j))))
        .toList();
System.out.println(pairs);
```

▶ Output

```
[S-red, S-blue, S-black, M-red, M-blue, M-black]
[(2,6), (7,1), (5,3)]
```

Q864 SA-015

• Basic

Split a list of sentences into distinct, lower-case words.

⚡ Quick answer

`flatMap` each sentence into words with `Arrays.stream(s.split(regex))` , normalise case, drop empties, then `distinct()` . `\\W+` splits on any run of non-word characters, which handles punctuation.

Java

```
List<String> reviews = List.of(
        "Fast delivery, great price!",
        "Great quality; fast refund.",
        "Price was GREAT.");
```


© TechTribers. For personal use only. Do not redistribute.

```
List<String> vocabulary = reviews.stream()
        .flatMap(r -> Arrays.stream(r.split("\\W+")))
        .map(w -> w.toLowerCase(Locale.ROOT))
        .filter(w -> !w.isEmpty())
        .distinct()
        .sorted()
        .toList();

System.out.println(vocabulary);
```

▶ Output

```
[delivery, fast, great, price, quality, refund, was]
```

💡 **Pro tip**

`Pattern.compile("\\W+").splitAsStream(r)` avoids building an intermediate array and reuses one compiled pattern if you keep it in a constant.

Q865 SA-016 • Intermediate

Convert a list of objects to a `Map`, handling duplicate keys with a merge rule.

⚡ **Quick answer**

`toMap(keyMapper, valueMapper, mergeFunction, mapSupplier)`. The **merge function** decides what happens on a duplicate key (keep first, keep latest, combine); without it you get `IllegalStateException`. `toMap` also throws `NullPointerException` for a **null value**, because it uses `Map.merge`.

See Q564 for the full list of `toMap` pitfalls; the four-argument form above avoids all of them except null values.

Java

```
record Product(String sku, String name, int price) { }

List<Product> feed = List.of(
        new Product("A1", "Kettle", 1499),
        new Product("B2", "Toaster", 2199),
        new Product("A1", "Kettle (new price)", 1299));

Map<String, Product> latestBySku = feed.stream()
        .collect(Collectors.toMap(Product::sku, Function.identity(),
                (older, newer) -> newer, LinkedHashMap::new));

Map<String, Integer> lowestPrice = feed.stream()
        .collect(Collectors.toMap(Product::sku, Product::price, Math::min, TreeMap::new));

try {
    feed.stream().collect(Collectors.toMap(Product::sku, Product::name));
} catch (IllegalStateException e) {
    System.out.println(e.getMessage());
```


© TechTribers. For personal use only. Do not redistribute.

```
}
latestBySku.values().forEach(System.out::println);
System.out.println(lowestPrice);
```

▶ Output

```
Duplicate key A1 (attempted merging values Kettle and Kettle (new price))
Product[sku=A1, name=Kettle (new price), price=1299]
Product[sku=B2, name=Toaster, price=2199]
{A1=1299, B2=2199}
```

🤝 Q866 SA-017

• Intermediate

Combine two lists into a map by position (zip keys with values).

⚡ Quick answer

Java has no `zip`, so stream the **indexes** and look up both lists: `IntStream.range(0, n).boxed().collect(toMap(keys::get, values::get))`. Check the sizes first, or zip up to the shorter one deliberately.

Java

```
List<String> headers = List.of("id", "name", "city", "joined");
List<String> row = List.of("42", "Asha", "Pune", "2019-04-01");

if (headers.size() != row.size()) throw new IllegalArgumentException("column count mismatch");

Map<String, String> record = IntStream.range(0, headers.size()).boxed()
        .collect(Collectors.toMap(headers::get, row::get, (a, b) -> b, LinkedHashMap::new));

System.out.println(record);
```

▶ Output

```
{id=42, name=Asha, city=Pune, joined=2019-04-01}
```

🏷️ Q867 SA-018

• Basic

Turn a map into a sorted list of `"key=value"` strings.

⚡ Quick answer

Stream the entry set, sort with `Map.Entry.comparingByKey()` (or `comparingByValue()`), and map each entry to a string. Sorting is essential because `HashMap` iteration order is unspecified.

© TechTribers. For personal use only. Do not redistribute.

```java
Map<String, Integer> config = Map.of("timeout", 30, "retries", 3, "pool", 10, "batch", 500);

List<String> byKey = config.entrySet().stream()
        .sorted(Map.Entry.comparingByKey())
        .map(e -> e.getKey() + "=" + e.getValue())
        .toList();

String byValueDesc = config.entrySet().stream()
        .sorted(Map.Entry.<String, Integer>comparingByValue().reversed())
        .map(e -> e.getKey() + "=" + e.getValue())
        .collect(Collectors.joining(", ", "{", "}"));

System.out.println(byKey);
System.out.println(byValueDesc);
```

▶ Output

```
[batch=500, pool=10, retries=3, timeout=30]
{batch=500, timeout=30, pool=10, retries=3}
```

⚠️ **Interview trap**

`Map.Entry.comparingByValue().reversed()` without the type witness `<String, Integer>` doesn't compile: the reversal stops target typing (the same inference rule as Q746).

🔄 Q868 SA-019 • Basic

Convert between `int[]`, `Integer[]` and `List<Integer>` with streams.

⚡ **Quick answer**

`Arrays.stream(int[]).boxed().toList()` goes from primitives to a list; `list.stream().mapToInt(Integer::intValue).toArray()` comes back. `Integer[]` uses `toArray(Integer[]::new)`. `Arrays.asList(int[])` is the classic trap: it gives a `List<int[]>` with **one** element.

```java
int[] primitives = {5, 3, 9};

List<Integer> list = Arrays.stream(primitives).boxed().toList();
Integer[] boxed = list.toArray(Integer[]::new);
int[] back = list.stream().mapToInt(Integer::intValue).toArray();

List<int[]> trap = Arrays.asList(primitives);

System.out.println(list + " " + Arrays.toString(boxed) + " " + Arrays.toString(back));
System.out.println("Arrays.asList(int[]) size = " + trap.size());
```


© TechTribers. For personal use only. Do not redistribute.

▶ Output

```
[5, 3, 9] [5, 3, 9] [5, 3, 9]
Arrays.asList(int[]) size = 1
```

Q869 SA-020 • Intermediate

Compute sliding windows of size k and a running (prefix) sum.

⚡ Quick answer

Windows: `IntStream.rangeClosed(0, n - k).mapToObj(i -> list.subList(i, i + k))`. Running sum: a stateless stream can't carry state between elements, so use `Arrays.parallelPrefix` on an array, or Java 24's `Gatherers.scan` (Q921). Recomputing each prefix with `subList(0, i).sum()` works but is O(n²).

Java

```
List<Integer> dailySales = List.of(4, 8, 6, 3, 9, 7);
int k = 3;

List<List<Integer>> windows = IntStream.rangeClosed(0, dailySales.size() - k)
        .mapToObj(i -> dailySales.subList(i, i + k))
        .toList();

List<Double> movingAvg = windows.stream()
        .map(w -> w.stream().mapToInt(Integer::intValue).average().orElse(0))
        .toList();

int[] running = dailySales.stream().mapToInt(Integer::intValue).toArray();
Arrays.parallelPrefix(running, Integer::sum);

System.out.println(windows);
System.out.println(movingAvg);
System.out.println(Arrays.toString(running));
```

▶ Output

```
[[4, 8, 6], [8, 6, 3], [6, 3, 9], [3, 9, 7]]
[6.0, 5.666666666666667, 6.0, 6.333333333333333]
[4, 12, 18, 21, 30, 37]
```

⚠ Interview trap

`subList` returns **views** of the original list. If the source is mutable and changes later, the windows change too; copy with `List.copyOf` when they must be stable.

© TechTribers. For personal use only. Do not redistribute.

Q870 SA-021

• Basic

Pair each element with its index (zipWithIndex) and print a numbered list.

⚡ Quick answer

Stream indexes and map to `"(i+1). value"`, or build small records holding index and value. Avoid an external `AtomicInteger` counter: it works sequentially but breaks with parallel streams and hides intent.

Java

```
record Indexed<T>(int index, T value) { }

List<String> steps = List.of("checkout", "pay", "confirm", "ship");

List<String> numbered = IntStream.range(0, steps.size())
        .mapToObj(i -> (i + 1) + ". " + steps.get(i))
        .toList();

List<Indexed<String>> indexed = IntStream.range(0, steps.size())
        .mapToObj(i -> new Indexed<>(i, steps.get(i)))
        .filter(x -> x.index() % 2 == 0)
        .toList();

numbered.forEach(System.out::println);
System.out.println(indexed);
```

▶ Output

```
1. checkout
2. pay
3. confirm
4. ship
[Indexed[index=0, value=checkout], Indexed[index=2, value=confirm]]
```

Q871 SA-022

• Advanced

Expand each order into its line items with `mapMulti` (Java 16), and compare it with `flatMap`.

⚡ Quick answer

`mapMulti((element, downstream) -> ...)` lets you push **zero or more** results per element into a `Consumer`, without creating an intermediate stream per element as `flatMap` does. It's handy when most elements produce 0 or 1 results, or when the expansion logic is imperative (loops, conditions).

Java

```
record Line(String sku, int qty) { }
record Order(String id, List<Line> lines) { }
```


© TechTribers. For personal use only. Do not redistribute.

```
List<Order> orders = List.of(
        new Order("O-1", List.of(new Line("pen", 3), new Line("pad", 0))),
        new Order("O-2", List.of()),
        new Order("O-3", List.of(new Line("ink", 2))));

List<String> pickList = orders.stream()
        .<String>mapMulti((order, out) -> {
            for (Line l : order.lines()) {
                if (l.qty() > 0) out.accept(order.id() + ":" + l.sku() + "x" + l.qty());
            }
        })
        .toList();

List<String> sameWithFlatMap = orders.stream()
        .flatMap(o -> o.lines().stream()
                .filter(l -> l.qty() > 0)
                .map(l -> o.id() + ":" + l.sku() + "x" + l.qty()))
        .toList();

System.out.println(pickList);
System.out.println(pickList.equals(sameWithFlatMap));
```

▶ Output

```
[O-1:penx3, O-3:inkx2]
true
```

⚠️ **Interview trap**

The type witness `.<String>mapMulti(...)` is usually required: javac can't infer the output type from a lambda that only calls `out.accept`.

© TechTribers. For personal use only. Do not redistribute.

Section 03

Sorting, Distinct, Limits and Ranking

Multi-key sorting, order-preserving distinct and distinct-by-key, finding duplicates, max and min with ties, Nth largest and second highest done correctly, top-N, pagination, sorting maps by value and lists by frequency, and merging sorted lists.

Questions 872–882 · 11 questions



Q872 SA-023

• Basic

Sort numbers in descending order, and sort strings by length then alphabetically.

⚡ Quick answer

`sorted(Comparator.reverseOrder())` for descending. For multiple keys, compose comparators: `comparingInt(String::length).thenComparing(Comparator.naturalOrder())`. `sorted()` is **stable**, so equal elements keep their encounter order.

Java

```
List<Integer> scores = List.of(42, 97, 13, 97, 58);
List<String> words = List.of("kiwi", "fig", "banana", "apple", "date", "plum");

List<Integer> desc = scores.stream().sorted(Comparator.reverseOrder()).toList();

List<String> byLenThenAlpha = words.stream()
        .sorted(Comparator.comparingInt(String::length).thenComparing(Comparator.naturalOrder()))
        .toList();

List<String> byLenDesc = words.stream()
        .sorted(Comparator.comparingInt(String::length).reversed()
                .thenComparing(Comparator.naturalOrder()))
        .toList();

System.out.println(desc);
System.out.println(byLenThenAlpha);
System.out.println(byLenDesc);
```

▶ Output

```
[97, 97, 58, 42, 13]
[fig, date, kiwi, plum, apple, banana]
[banana, apple, date, kiwi, plum, fig]
```


© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

`comparing(...).thenComparing(...).reversed()` reverses **every** key. To reverse only one key, put `.reversed()` right after that key's comparator, as in `byLenDesc`.

Q873 SA-024 • Intermediate

Get distinct elements by a property (distinct-by-key), keeping the first occurrence.

⚡ **Quick answer**

`distinct()` keeps the **first occurrence** of each element, in encounter order, using `equals` / `hashCode`. For distinct-**by-key** there's no built-in operator: collect to a `LinkedHashMap` keyed by the property with `putIfAbsent` semantics (`toMap` with a keep-first merge), or use a `TreeSet` with a comparator on the key.

Java

```
record Customer(String email, String name, String city) { }

List<Integer> ids = List.of(5, 3, 5, 8, 3, 1);
System.out.println(ids.stream().distinct().toList());

List<Customer> signups = List.of(
        new Customer("a@x.in", "Asha", "Pune"),
        new Customer("r@x.in", "Ravi", "Delhi"),
        new Customer("A@X.IN", "Asha K", "Pune"),
        new Customer("m@x.in", "Meera", "Mumbai"),
        new Customer("r@x.in", "Ravi S", "Delhi"));

Collection<Customer> uniqueByEmail = signups.stream()
        .collect(Collectors.toMap(c -> c.email().toLowerCase(Locale.ROOT), c -> c,
                (first, dup) -> first, LinkedHashMap::new))
        .values();

uniqueByEmail.forEach(c -> System.out.println(c.name() + " <" + c.email() + ">"));
```

▶ **Output**

```
[5, 3, 8, 1]
Asha <a@x.in>
Ravi <r@x.in>
Meera <m@x.in>
```

⚠️ **Interview trap**

The popular `filter(seen::add)` trick with a `HashSet` works only sequentially; in a parallel stream the shared set is a data race. The collector version is safe either way.

© TechTribers. For personal use only. Do not redistribute.

Q874 SA-025 • Basic

Find the duplicate elements in a list (and how many times each occurs).

⚡ Quick answer

Group by identity with `counting()` and keep entries whose count is above 1. This reports each duplicate **once**, with its count, in O(n). The `Set.add` trick (`filter(x -> !seen.add(x))`) is shorter but stateful, and reports an element once per extra occurrence.

Note the second output: `A1` appears twice because it occurs three times. If the question is "which elements are duplicated", follow it with `distinct()` .

Java

```
List<String> skus = List.of("A1", "B2", "A1", "C3", "B2", "A1", "D4");

Map<String, Long> dupCounts = skus.stream()
        .collect(Collectors.groupingBy(Function.identity(), TreeMap::new, Collectors.counting()))
        .entrySet().stream()
        .filter(e -> e.getValue() > 1)
        .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue,
                (a, b) -> a, TreeMap::new));

Set<String> seen = new HashSet<>();
List<String> viaSetAdd = skus.stream().filter(s -> !seen.add(s)).toList();

System.out.println(dupCounts);
System.out.println(viaSetAdd);
```

▶ Output

```
{A1=3, B2=2}
[A1, B2, A1]
```

Q875 SA-026 • Basic

Find the maximum and minimum, and the longest string, handling ties.

⚡ Quick answer

`max` / `min` take a comparator and return `Optional` . On **ties**, the current JDK's `max` keeps the **first** of the tied elements in encounter order (it's implemented with `BinaryOperator.maxBy` , which keeps the earlier value when the comparator says equal), as the output shows with `stream` ; the Javadoc doesn't promise this. `reduce((a, b) -> ...)` lets you choose explicitly. To return **all** tied winners, compute the maximum key first, then filter.

Java

```
List<String> words = List.of("java", "stream", "kotlin", "go", "python");
```


© TechTribers. For personal use only. Do not redistribute.

```
String maxByLength = words.stream().max(Comparator.comparingInt(String::length)).orElseThrow();
String firstLongest = words.stream()
        .reduce((a, b) -> b.length() > a.length() ? b : a)
        .orElseThrow();

int longest = words.stream().mapToInt(String::length).max().orElse(0);
List<String> allLongest = words.stream().filter(w -> w.length() == longest).toList();

IntSummaryStatistics stats = List.of(17, 4, 42, 9).stream()
        .mapToInt(Integer::intValue).summaryStatistics();

System.out.println(maxByLength + " | " + firstLongest + " | " + allLongest);
System.out.println("min=" + stats.getMin() + " max=" + stats.getMax());
```

▶ Output

```
stream | stream | [stream, kotlin, python]
min=4 max=42
```

⚠️ Interview trap

Relying on which tied element `max` returns. The Javadoc doesn't specify it; write the tie-break into the comparator (`.thenComparing(...)`) if it matters.

Q876 SA-027 • Intermediate

Find the Nth largest distinct number (for example, the 3rd largest).

⚡ Quick answer

`distinct()`, sort descending, `skip(n - 1)`, `findFirst()`. Without `distinct()`, duplicates shift the answer: in `[50, 50, 40, 30]` the "2nd largest" would come out as 50. Return an `Optional` so "fewer than N distinct values" is explicit.

Java

```
static Optional<Integer> nthLargest(List<Integer> values, int n) {
    return values.stream()
            .distinct()
            .sorted(Comparator.reverseOrder())
            .skip(n - 1L)
            .findFirst();
}

List<Integer> marks = List.of(88, 95, 70, 95, 88, 61, 99);
System.out.println("1st: " + nthLargest(marks, 1).orElseThrow());
System.out.println("3rd: " + nthLargest(marks, 3).orElseThrow());
System.out.println("9th: " + nthLargest(marks, 9));
```

▶ Output

```
1st: 99
3rd: 88
```


© TechTribers. For personal use only. Do not redistribute.

```
9th: Optional.empty
```

> 💡 **Pro tip**
> For huge inputs and small N, sorting everything is wasteful: keep a min-heap of size N (`PriorityQueue`), which is O(n log N). See Q894 for a top-K collector.

🥈 Q877 SA-028 • Basic

Find the second highest number, and explain the duplicate trap.

> ⚡ **Quick answer**
> `stream().distinct().sorted(reverseOrder()).skip(1).findFirst()` . Interviewers check whether you handle **duplicates of the maximum** (`[90, 90, 75]` should give 75) and **lists with fewer than two distinct values** (return empty, don't throw).

Java

```java
List<Integer> withTies = List.of(90, 75, 90, 60);
List<Integer> single = List.of(42, 42);

Optional<Integer> correct = withTies.stream().distinct()
        .sorted(Comparator.reverseOrder()).skip(1).findFirst();
Optional<Integer> wrong = withTies.stream()
        .sorted(Comparator.reverseOrder()).skip(1).findFirst();
Optional<Integer> none = single.stream().distinct()
        .sorted(Comparator.reverseOrder()).skip(1).findFirst();

System.out.println("with distinct: " + correct.orElseThrow());
System.out.println("without distinct: " + wrong.orElseThrow());
System.out.println("only one distinct value: " + none);
```

▶ **Output**

```
with distinct: 75
without distinct: 90
only one distinct value: Optional.empty
```

🔝 Q878 SA-029 • Basic

Get the top N and bottom N elements by a score.

> ⚡ **Quick answer**
> Sort by the score (descending for top, ascending for bottom) and `limit(n)` . Add a secondary key so ties are deterministic.

© TechTribers. For personal use only. Do not redistribute.

```java
record Player(String name, int points) { }

List<Player> board = List.of(new Player("Asha", 1820), new Player("Ravi", 1675),
        new Player("Meera", 1910), new Player("Karan", 1675), new Player("Neha", 1490));

List<String> top3 = board.stream()
        .sorted(Comparator.comparingInt(Player::points).reversed().thenComparing(Player::name))
        .limit(3).map(Player::name).toList();

List<String> bottom2 = board.stream()
        .sorted(Comparator.comparingInt(Player::points).thenComparing(Player::name))
        .limit(2).map(Player::name).toList();

System.out.println(top3 + " " + bottom2);
```

▶ Output

```
[Meera, Asha, Karan] [Neha, Karan]
```

Q879 SA-030 • Basic

Implement pagination with `skip` and `limit` (page number and page size).

⚡ Quick answer

Page `p` (1-based) of size `s` is `skip((long) (p - 1) * s).limit(s)`. Sort first so pages are stable, and compute the total page count with ceiling division. For database data, page in SQL instead (keyset pagination; see Q817).

```java
static <T> List<T> page(List<T> items, int page, int size) {
    return items.stream().skip((long) (page - 1) * size).limit(size).toList();
}

List<Integer> ids = IntStream.rangeClosed(101, 112).boxed().toList();
int size = 5;
int pages = (ids.size() + size - 1) / size;

for (int p = 1; p <= pages; p++) {
    System.out.println("page " + p + "/" + pages + ": " + page(ids, p, size));
}
System.out.println("page 9: " + page(ids, 9, size));
```

▶ Output

```
page 1/3: [101, 102, 103, 104, 105]
page 2/3: [106, 107, 108, 109, 110]
```


© TechTribers. For personal use only. Do not redistribute.

```
page 3/3: [111, 112]
page 9: []
```

📉 Q880 SA-031

• Intermediate

Sort a map by value (descending) and keep the result as an ordered map.

⚡ Quick answer

Stream the entries, sort with `Map.Entry.comparingByValue(Comparator.reverseOrder())`, and collect with `toMap(..., LinkedHashMap::new)`. A plain `toMap` would put the entries back into a `HashMap` and lose the sort.

Java

```
Map<String, Integer> pageViews = Map.of("home", 1520, "pricing", 870, "docs", 2310,
        "blog", 870, "careers", 120);

Map<String, Integer> ranked = pageViews.entrySet().stream()
        .sorted(Map.Entry.<String, Integer>comparingByValue(Comparator.reverseOrder())
                .thenComparing(Map.Entry.comparingByKey()))
        .collect(Collectors.toMap(Map.Entry::getKey, Map.Entry::getValue,
                (a, b) -> a, LinkedHashMap::new));

System.out.println(ranked);
```

▶ Output

```
{docs=2310, home=1520, blog=870, pricing=870, careers=120}
```

📶 Q881 SA-032

• Intermediate

Sort elements by frequency (most frequent first), breaking ties by value.

⚡ Quick answer

Count frequencies once into a map, then sort the **distinct** elements with a comparator that reads the counts. Calling `Collections.frequency` inside the comparator recounts the whole list on every comparison: O(n² log n).

Java

```
List<Integer> clicks = List.of(4, 6, 2, 6, 4, 6, 9, 2, 4, 3);

Map<Integer, Long> freq = clicks.stream()
        .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()));

List<Integer> byFrequency = freq.keySet().stream()
```


© TechTribers. For personal use only. Do not redistribute.

```
        .sorted(Comparator.comparing((Integer x) -> freq.get(x)).reversed()
                .thenComparing(Comparator.naturalOrder()))
        .toList();

List<Integer> expanded = byFrequency.stream()
        .flatMap(x -> Collections.nCopies(freq.get(x).intValue(), x).stream())
        .toList();

System.out.println(byFrequency);
System.out.println(expanded);
```

▶ Output

```
[4, 6, 2, 3, 9]
[4, 4, 4, 6, 6, 6, 2, 2, 3, 9]
```

Q882 SA-033 • Basic

Merge two lists, remove duplicates and sort; how does `Stream.concat` behave?

⚡ Quick answer

`Stream.concat(a.stream(), b.stream())` produces a lazily concatenated stream; follow with `distinct()` and `sorted()`. For more than two sources use `Stream.of(a, b, c).flatMap(List::stream)`, because deeply nested `concat` calls can cause a `StackOverflowError`.

Java

```
List<Integer> warehouseA = List.of(5, 1, 9, 3);
List<Integer> warehouseB = List.of(3, 7, 1, 11);
List<Integer> warehouseC = List.of(2, 9);

List<Integer> merged = Stream.concat(warehouseA.stream(), warehouseB.stream())
        .distinct().sorted().toList();

List<Integer> allThree = Stream.of(warehouseA, warehouseB, warehouseC)
        .flatMap(List::stream)
        .distinct().sorted().toList();

System.out.println(merged);
System.out.println(allThree);
```

▶ Output

```
[1, 3, 5, 7, 9, 11]
[1, 2, 3, 5, 7, 9, 11]
```

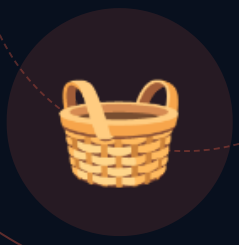

© TechTribers. For personal use only. Do not redistribute.

💡 **Pro tip**

When both inputs are already sorted and large, a classic two-pointer merge is O(n+m) instead of the O((n+m) log(n+m)) sort; streams don't have a merge operator.

Section 04

Collectors: Grouping, Partitioning and Reducing



groupingBy with sorted keys and values, frequency maps, partitioning with counts, mapping and joining downstream, multi-level grouping, summary statistics, toMap with merging, collectingAndThen, teeing, filtering and flatMapping, reduce, a custom top-K collector, and Integer counts and EnumMaps.

Questions 883–895 · 13 questions

Q883 SA-034 • Basic

Group words by length, with sorted keys and alphabetically sorted groups.

⚡ **Quick answer**

`groupingBy(String::length, TreeMap::new, toList())` gives sorted keys; for sorted values, either sort the stream **before** grouping (grouping preserves encounter order) or use a `TreeSet` downstream (which also removes duplicates).

Java

```
List<String> words = List.of("java", "go", "rust", "kotlin", "c", "scala", "dart", "swift");

Map<Integer, List<String>> byLength = words.stream()
        .sorted()
        .collect(Collectors.groupingBy(String::length, TreeMap::new, Collectors.toList()));

Map<Integer, Long> countByLength = words.stream()
        .collect(Collectors.groupingBy(String::length, TreeMap::new, Collectors.counting()));

System.out.println(byLength);
System.out.println(countByLength);
```

▶ **Output**

```
{1=[c], 2=[go], 4=[dart, java, rust], 5=[scala, swift], 6=[kotlin]}
{1=1, 2=1, 4=3, 5=2, 6=1}
```


© TechTribers. For personal use only. Do not redistribute.

`groupingBy(classifier, mapFactory, downstream)`

classifier	`String::length` — the KEY of each group
mapFactory	`TreeMap::new` — order of keys (default HashMap)
downstream	`counting()`, `mapping(...)`, ... — what each group's VALUE becomes

Downstreams: toList · toSet · counting · summingInt · averagingInt · maxBy · mapping · filtering · flatMapping · joining · collectingAndThen · teeing · summarizingInt · another groupingBy

---

Q884 SA-035 • Intermediate

Choose the container for each group: `toSet`, `toCollection(TreeSet::new)` or `toUnmodifiableList`.

⚡ **Quick answer**

The downstream collector decides the type of each group's value. `toList()` keeps duplicates and encounter order; `toSet()` removes duplicates but has no order; `toCollection(TreeSet::new)` removes duplicates **and** sorts; `toUnmodifiableList()` protects the result from callers. Pick it deliberately instead of post-processing every group.

Java

```
record Visit(String user, String page) { }

List<Visit> visits = List.of(new Visit("asha", "/pricing"), new Visit("ravi", "/docs"),
        new Visit("asha", "/docs"), new Visit("asha", "/pricing"), new Visit("ravi", "/blog"));

Map<String, List<String>> allPages = visits.stream()
        .collect(Collectors.groupingBy(Visit::user, TreeMap::new,
                Collectors.mapping(Visit::page, Collectors.toList())));

Map<String, TreeSet<String>> distinctSorted = visits.stream()
        .collect(Collectors.groupingBy(Visit::user, TreeMap::new,
                Collectors.mapping(Visit::page, Collectors.toCollection(TreeSet::new))));

Map<String, List<String>> readOnly = visits.stream()
        .collect(Collectors.groupingBy(Visit::user, TreeMap::new,
                Collectors.mapping(Visit::page, Collectors.toUnmodifiableList())));

System.out.println(allPages);
System.out.println(distinctSorted);
try { readOnly.get("asha").add("/hack"); }
catch (UnsupportedOperationException e) { System.out.println("unmodifiable group: " + e); }
```

▶ Output

© TechTribers. For personal use only. Do not redistribute.

```
{asha=[/pricing, /docs, /pricing], ravi=[/docs, /blog]}
{asha=[/docs, /pricing], ravi=[/blog, /docs]}
unmodifiable group: java.lang.UnsupportedOperationException
```

Q885 SA-036 • Basic

Partition numbers into even and odd, and count each side.

⚡ **Quick answer**

`partitioningBy(predicate)` always returns a map with **both** keys `false` and `true`, even if one side is empty, which `groupingBy` doesn't guarantee. It accepts a downstream collector such as `counting()` or `summingInt`.

See Q563 for when to choose `partitioningBy` over `groupingBy`.

Java

```java
List<Integer> numbers = List.of(3, 8, 15, 4, 23, 42, 7);

Map<Boolean, List<Integer>> evenOdd = numbers.stream()
        .collect(Collectors.partitioningBy(n -> n % 2 == 0));

Map<Boolean, Long> counts = numbers.stream()
        .collect(Collectors.partitioningBy(n -> n % 2 == 0, Collectors.counting()));

Map<Boolean, List<Integer>> allSmall = numbers.stream()
        .collect(Collectors.partitioningBy(n -> n > 100));

System.out.println(evenOdd);
System.out.println(counts);
System.out.println(allSmall);
```

▶ **Output**

```
{false=[3, 15, 23, 7], true=[8, 4, 42]}
{false=4, true=3}
{false=[3, 8, 15, 4, 23, 42, 7], true=[]}
```

Q886 SA-037 • Intermediate

Group names by first letter and join each group into one comma-separated string.

⚡ **Quick answer**

Use a **downstream** collector: `groupingBy(classifier, mapping(mapper, joining(", ")))`. `mapping` transforms elements inside each group before the next collector sees them.

© TechTribers. For personal use only. Do not redistribute.

```java
List<String> names = List.of("Asha", "Ravi", "Arjun", "Meera", "Rahul", "Ananya", "Mohan");

Map<Character, String> byInitial = names.stream()
        .sorted()
        .collect(Collectors.groupingBy(n -> n.charAt(0), TreeMap::new,
                Collectors.mapping(String::toUpperCase, Collectors.joining(", "))));

byInitial.forEach((letter, joined) -> System.out.println(letter + " -> " + joined));
```

▶ Output

```
A -> ANANYA, ARJUN, ASHA
M -> MEERA, MOHAN
R -> RAHUL, RAVI
```

Q887 SA-038 • Intermediate

Build a two-level grouping: department, then city, then the list of names.

⚡ Quick answer

Nest `groupingBy` as the downstream of another `groupingBy`, ending with `mapping(Employee::name, toList())`. Use `TreeMap` at each level when the output is shown to people or compared in tests.

```java
Map<String, Map<String, List<String>>> deptCityNames = employees.stream()
        .collect(Collectors.groupingBy(Employee::dept, TreeMap::new,
                Collectors.groupingBy(Employee::city, TreeMap::new,
                        Collectors.mapping(Employee::name, Collectors.toList()))));

deptCityNames.forEach((dept, byCity) -> System.out.println(dept + " " + byCity));
```

▶ Output

```
Engineering {Bengaluru=[Ravi], Pune=[Asha, Vikram]}
Finance {Mumbai=[Divya], Pune=[Sneha]}
HR {Bengaluru=[Arjun], Delhi=[Neha]}
Sales {Delhi=[Rahul], Mumbai=[Meera], Pune=[Karan]}
```

💡 Pro tip

Beyond two levels, nested maps become hard to use; consider grouping by a composite key record instead: `groupingBy(e -> new DeptCity(e.dept(), e.city()))`.

© TechTribers. For personal use only. Do not redistribute.

Q888 SA-039

• Basic

Compute count, sum, min, max and average in one pass with summary statistics.

⚡ Quick answer

`IntSummaryStatistics` (from `mapToInt(...).summaryStatistics()` or `Collectors.summarizingInt`) computes count, sum, min, max and average in **one pass**. Calling `sum()`, `max()` and `average()` separately would stream the data three times.

Java

```
List<Integer> responseMs = List.of(120, 85, 430, 97, 210, 64, 155);

IntSummaryStatistics s = responseMs.stream().mapToInt(Integer::intValue).summaryStatistics();
System.out.println(s);
System.out.printf("avg %.1f ms, max %d ms%n", s.getAverage(), s.getMax());

double avg = responseMs.stream().collect(Collectors.averagingInt(Integer::intValue));
int total = responseMs.stream().collect(Collectors.summingInt(Integer::intValue));
System.out.println(avg + " " + total);

IntSummaryStatistics empty = IntStream.empty().summaryStatistics();
System.out.println("empty: min=" + empty.getMin() + " avg=" + empty.getAverage());
```

▶ Output

```
IntSummaryStatistics{count=7, sum=1161, min=64, average=165.857143, max=430}
avg 165.9 ms, max 430 ms
165.85714285714286 1161
empty: min=2147483647 avg=0.0
```

Which collector for which question?

How many per key?	`groupingBy(k, counting())`
Sum / average per key?	`groupingBy(k, summingInt / averagingInt)`
Count, min, max, avg at once?	`summarizingInt / summaryStatistics()`
Best element per key?	`groupingBy(k, collectingAndThen(maxBy(c), ...))`
Two groups, always both keys?	`partitioningBy(predicate, downstream)`
Key -> single value?	`toMap(k, v, merge, LinkedHashMap::new)`
Two results in one pass?	`teeing(c1, c2, merger)`
Top K in O(K) memory?	`custom Collector.of(PriorityQueue ...)`

⚠️ Interview trap

On an empty input, `getMin()` returns `Integer.MAX_VALUE` and `getMax()` returns `Integer.MIN_VALUE`. Check `getCount()` before using them.

© TechTribers. For personal use only. Do not redistribute.

➕ Q889 SA-040

• Intermediate

Sum values per key with `toMap` merging (aggregate order amounts by customer).

⚡ Quick answer

`toMap(key, value, Integer::sum, TreeMap::new)` adds values that share a key. The equivalent `groupingBy(key, summingInt(value))` is often clearer; `toMap` is handy when the merge isn't a standard collector.

Java

```
record OrderLine(String customer, int amount) { }

List<OrderLine> lines = List.of(new OrderLine("asha", 1200), new OrderLine("ravi", 450),
        new OrderLine("asha", 300), new OrderLine("meera", 999), new OrderLine("ravi", 1550));

Map<String, Integer> viaToMap = lines.stream()
        .collect(Collectors.toMap(OrderLine::customer, OrderLine::amount,
                Integer::sum, TreeMap::new));

Map<String, Integer> viaGrouping = lines.stream()
        .collect(Collectors.groupingBy(OrderLine::customer, TreeMap::new,
                Collectors.summingInt(OrderLine::amount)));

System.out.println(viaToMap);
System.out.println(viaToMap.equals(viaGrouping));
```

▶ Output

```
{asha=1500, meera=999, ravi=2000}
true
```

🎁 Q890 SA-041

• Intermediate

Use `collectingAndThen` to return an unmodifiable list and to unwrap `maxBy` results.

⚡ Quick answer

`collectingAndThen(downstream, finisher)` applies a function to the collector's result. Common finishers: `Collections::unmodifiableList`, `Optional::orElseThrow` after `maxBy` / `minBy` inside `groupingBy` (where each group is non-empty by construction), and `List::size`.

Java

```
record Book(String title, String genre, int pages) { }

List<Book> books = List.of(new Book("Dune", "SF", 612), new Book("Emma", "Classic", 474),
```


© TechTribers. For personal use only. Do not redistribute.

```
        new Book("Neuromancer", "SF", 271), new Book("Persuasion", "Classic", 249));

List<String> titles = books.stream().map(Book::title)
        .collect(Collectors.collectingAndThen(Collectors.toList(), Collections::unmodifiableList));

Map<String, String> longestPerGenre = books.stream()
        .collect(Collectors.groupingBy(Book::genre, TreeMap::new,
                Collectors.collectingAndThen(
                        Collectors.maxBy(Comparator.comparingInt(Book::pages)),
                        b -> b.orElseThrow().title())));

System.out.println(titles);
System.out.println(longestPerGenre);
```

▶ Output

```
[Dune, Emma, Neuromancer, Persuasion]
{Classic=Emma, SF=Dune}
```

💡 **Pro tip**

Since Java 10, `Collectors.toUnmodifiableList()` and `Stream.toList()` (Java 16) are simpler ways to get an unmodifiable list.

Q891 SA-042 • Intermediate

Compute two results in one pass with `Collectors.teeing` (Java 12).

⚡ **Quick answer**

`teeing(collector1, collector2, merger)` feeds every element to two collectors and combines their results. Typical uses: min and max together, count and sum for a custom average, or "total vs filtered" ratios, without streaming the data twice.

See Q359 for the API background; this question is about using it to replace two passes.

Java

```
record Range(int min, int max) { }

List<Integer> temps = List.of(31, 27, 35, 29, 38, 26);

Range range = temps.stream().collect(Collectors.teeing(
        Collectors.minBy(Comparator.<Integer>naturalOrder()),
        Collectors.maxBy(Comparator.<Integer>naturalOrder()),
        (min, max) -> new Range(min.orElseThrow(), max.orElseThrow())));

String hotShare = temps.stream().collect(Collectors.teeing(
        Collectors.counting(),
        Collectors.filtering(t -> t >= 35, Collectors.counting()),
        (all, hot) -> hot + " of " + all + " days were 35+"));
```


© TechTribers. For personal use only. Do not redistribute.

```
System.out.println(range + " | " + hotShare);
```

▶ Output

```
Range[min=26, max=38] | 2 of 6 days were 35+
```

⚠️ Interview trap

Without the type witnesses `Comparator.<Integer>naturalOrder()`, this fails to compile: javac can't infer the element type through `minBy` / `maxBy` nested inside `teeing`, and reports "inference variable T has incompatible upper bounds". Our first version hit exactly that.

Q892 SA-043 • Advanced

Use the `filtering` and `flatMapping` downstream collectors (Java 9).

⚡ Quick answer

`filtering(predicate, downstream)` filters **inside each group**, so groups with no matches still appear with an empty value; filtering **before** `groupingBy` drops those groups entirely. `flatMapping(mapper, downstream)` flattens a collection per element inside each group.

Java

```
record Ticket(String team, String priority, List<String> tags) { }

List<Ticket> tickets = List.of(
        new Ticket("payments", "P1", List.of("db", "timeout")),
        new Ticket("payments", "P3", List.of("ui")),
        new Ticket("search", "P3", List.of("index")),
        new Ticket("search", "P2", List.of("index", "latency")));

Map<String, Long> p1PerTeam = tickets.stream()
        .collect(Collectors.groupingBy(Ticket::team, TreeMap::new,
                Collectors.filtering(t -> t.priority().equals("P1"), Collectors.counting())));

Map<String, Long> filteredFirst = tickets.stream()
        .filter(t -> t.priority().equals("P1"))
        .collect(Collectors.groupingBy(Ticket::team, TreeMap::new, Collectors.counting()));

Map<String, Set<String>> tagsPerTeam = tickets.stream()
        .collect(Collectors.groupingBy(Ticket::team, TreeMap::new,
                Collectors.flatMapping(t -> t.tags().stream(),
                        Collectors.toCollection(TreeSet::new))));

System.out.println(p1PerTeam + " vs " + filteredFirst);
System.out.println(tagsPerTeam);
```

▶ Output

© TechTribers. For personal use only. Do not redistribute.

```
{payments=1, search=0} vs {payments=1}
{payments=[db, timeout, ui], search=[index, latency]}
```

Q893 SA-044 • Intermediate

Use `reduce` for a product and a sum of squares, and know when not to use it.

⚡ **Quick answer**

`reduce(identity, accumulator)` folds a stream into one value; the identity must be a true neutral element (0 for sum, 1 for product) and the accumulator **associative**, or parallel results will be wrong. For building strings or collections, use `collect` / `joining` instead: `reduce` with string concatenation creates a new string per element (O(n²) copying).

Java

```java
List<Integer> factors = List.of(2, 3, 4, 5);

int product = factors.stream().reduce(1, (a, b) -> a * b);
int sumOfSquares = factors.stream().map(x -> x * x).reduce(0, Integer::sum);
long bigProduct = LongStream.rangeClosed(1, 20).reduce(1L, (a, b) -> a * b);
Optional<Integer> noIdentity = Stream.<Integer>empty().reduce(Integer::sum);

String joined = Stream.of("a", "b", "c").collect(Collectors.joining("-"));

System.out.println(product + " " + sumOfSquares + " " + bigProduct);
System.out.println(noIdentity + " " + joined);
```

▶ **Output**

```
120 54 2432902008176640000
Optional.empty a-b-c
```

⚠️ **Interview trap**

`LongStream.rangeClosed(1, 21).reduce(1L, (a, b) -> a * b)` silently overflows (21! exceeds `Long.MAX_VALUE`). Use `BigInteger` for factorials beyond 20 (Q919).

© TechTribers. For personal use only. Do not redistribute.

Q894 SA-045

• Advanced

Write a custom `Collector` that keeps the top K elements (min-heap) in one pass.

⚡ Quick answer

Use `Collector.of(supplier, accumulator, combiner, finisher)`. The supplier creates a `PriorityQueue` ordered by the comparator (a min-heap); the accumulator adds and evicts the smallest when the size exceeds K; the combiner merges two heaps (needed for parallel streams); the finisher returns the elements sorted. Memory is O(K) instead of sorting all n elements.

The same design works for any streaming aggregation that isn't in `Collectors`: a percentile sketch, a bounded "most recent N", a histogram. Declare characteristics such as `UNORDERED` only when they're true; they allow the framework to skip ordering work.

For top-K per group, use it as a downstream collector: `groupingBy(Employee::dept, topK(2, comparingInt(Employee::salary)))` (see Q567 for top N per group).

Java

```
static <T> Collector<T, ?, List<T>> topK(int k, Comparator<? super T> cmp) {
    return Collector.of(
            () -> new PriorityQueue<T>(cmp),
            (heap, x) -> { heap.offer(x); if (heap.size() > k) heap.poll(); },
            (h1, h2) -> {
                for (T x : h2) { h1.offer(x); if (h1.size() > k) h1.poll(); }
                return h1;
            },
            heap -> heap.stream().sorted(cmp.reversed()).toList());
}

record Trade(String id, int qty) { }

List<Trade> trades = List.of(new Trade("t1", 40), new Trade("t2", 900), new Trade("t3", 15),
        new Trade("t4", 610), new Trade("t5", 88), new Trade("t6", 720));

List<Trade> biggest3 = trades.stream().collect(topK(3, Comparator.comparingInt(Trade::qty)));
List<Integer> top2 = IntStream.rangeClosed(1, 1_000_000).boxed()
        .collect(topK(2, Comparator.naturalOrder()));

System.out.println(biggest3);
System.out.println(top2);
```

▶ Output

```
[Trade[id=t2, qty=900], Trade[id=t6, qty=720], Trade[id=t4, qty=610]]
[1000000, 999999]
```


© TechTribers. For personal use only. Do not redistribute.

Q895 SA-046

• Intermediate

Get counts as `Integer` instead of `Long`, and group into an `EnumMap`.

⚡ **Quick answer**

`counting()` always produces `Long`. For `Integer` counts use `summingInt(x -> 1)` or `collectingAndThen(counting(), Long::intValue)`. For enum keys, pass `() -> new EnumMap<>(Type.class)` as the map factory: compact, fast and ordered by declaration.

Java

```
enum Level { DEBUG, INFO, WARN, ERROR }
record LogLine(Level level, String msg) { }

List<LogLine> logs = List.of(
        new LogLine(Level.INFO, "start"), new LogLine(Level.ERROR, "db down"),
        new LogLine(Level.WARN, "slow"), new LogLine(Level.INFO, "retry"),
        new LogLine(Level.ERROR, "db down"));

Map<Level, Integer> perLevel = logs.stream()
        .collect(Collectors.groupingBy(LogLine::level, () -> new EnumMap<>(Level.class),
                Collectors.summingInt(l -> 1)));

Map<Level, Integer> withZeros = new EnumMap<>(Level.class);
for (Level l : Level.values()) withZeros.put(l, perLevel.getOrDefault(l, 0));

System.out.println(perLevel);
System.out.println(withZeros);
```

▶ **Output**

```
{INFO=2, WARN=1, ERROR=2}
{DEBUG=0, INFO=2, WARN=1, ERROR=2}
```

💡 **Pro tip**

`groupingBy` only creates keys that occur. For dashboards that must show every level (including zero), pre-fill the map as above.

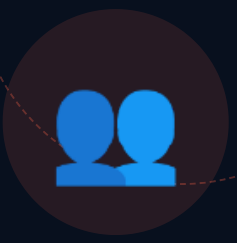
© TechTribers. For personal use only. Do not redistribute.

Section 05

Employee Dataset Scenarios

The most-asked interview family, all on one realistic dataset: grouping and counting by department, averages, highest-paid and second-highest salary with ties, top-N per department, city totals, date filters, age and gender statistics, immutable raises, partitions and per-department statistics.

Questions 896–907 · 12 questions



Q896 SA-047

• Basic

Group employees by department and list their names, sorted.

⚡ Quick answer

`groupingBy(Employee::dept, TreeMap::new, mapping(Employee::name, toList()))`, with the stream sorted by name first so each group's list is sorted. The dataset below is used by every question in this section.

Using a record keeps the dataset compact and gives correct `equals`, `hashCode` and `toString` for free, which matters when employees become map keys or appear in test assertions.

Java

```
record Employee(int id, String name, String dept, int salary, int age,
                String gender, String city, int joined) { }

List<Employee> employees = List.of(
        new Employee(1, "Asha", "Engineering", 185000, 31, "F", "Pune", 2019),
        new Employee(2, "Ravi", "Engineering", 142000, 28, "M", "Bengaluru", 2021),
        new Employee(3, "Meera", "Sales", 98000, 35, "F", "Mumbai", 2016),
        new Employee(4, "Karan", "Sales", 112000, 41, "M", "Pune", 2012),
        new Employee(5, "Neha", "HR", 76000, 26, "F", "Delhi", 2023),
        new Employee(6, "Vikram", "Engineering", 185000, 38, "M", "Pune", 2014),
        new Employee(7, "Divya", "Finance", 131000, 33, "F", "Mumbai", 2018),
        new Employee(8, "Arjun", "HR", 69000, 24, "M", "Bengaluru", 2024),
        new Employee(9, "Sneha", "Finance", 158000, 45, "F", "Pune", 2010),
        new Employee(10, "Rahul", "Sales", 104000, 29, "M", "Delhi", 2022));

Map<String, List<String>> namesByDept = employees.stream()
        .sorted(Comparator.comparing(Employee::name))
        .collect(Collectors.groupingBy(Employee::dept, TreeMap::new,
                Collectors.mapping(Employee::name, Collectors.toList())));

namesByDept.forEach((dept, names) -> System.out.println(dept + ": " + names));
```

▶ Output

© TechTribers. For personal use only. Do not redistribute.

```
Engineering: [Asha, Ravi, Vikram]
Finance: [Divya, Sneha]
HR: [Arjun, Neha]
Sales: [Karan, Meera, Rahul]
```

Q897 SA-048

• Basic

Count employees per department, and find the department with the most employees.

⚡ Quick answer

`groupingBy(dept, counting())`, then stream the entries and take `max(comparingByValue())`. On a tie, decide explicitly (here, the alphabetically first department wins) instead of relying on map iteration order.

Java

```java
Map<String, Long> headcount = employees.stream()
        .collect(Collectors.groupingBy(Employee::dept, TreeMap::new, Collectors.counting()));

Map.Entry<String, Long> largest = headcount.entrySet().stream()
        .max(Map.Entry.<String, Long>comparingByValue()
                .thenComparing(Map.Entry.comparingByKey(Comparator.reverseOrder())))
        .orElseThrow();

System.out.println(headcount);
System.out.println("largest: " + largest.getKey() + " (" + largest.getValue() + ")");
```

▶ Output

```
{Engineering=3, Finance=2, HR=2, Sales=3}
largest: Engineering (3)
```

Q898 SA-049

• Intermediate

Compute the average salary per department, rounded, highest first.

⚡ Quick answer

`groupingBy(dept, averagingInt(salary))` returns `Double` averages; sort the entries by value descending into a `LinkedHashMap`, and round for display only.

© TechTribers. For personal use only. Do not redistribute.

```java
Map<String, Double> avgSalary = employees.stream()
        .collect(Collectors.groupingBy(Employee::dept, Collectors.averagingInt(Employee::salary)));

avgSalary.entrySet().stream()
        .sorted(Map.Entry.<String, Double>comparingByValue().reversed())
        .forEach(e -> System.out.printf("%-12s %,10.0f%n", e.getKey(), e.getValue()));
```

▶ Output

```
Engineering     170,667
Finance         144,500
Sales           104,667
HR               72,500
```

💡 **Pro tip**

For money in real systems, use `BigDecimal` (or integer minor units such as paise) and `Collectors.reducing(BigDecimal.ZERO, ..., BigDecimal::add)`; `double` averages are fine for reporting, not for accounting.

👑 Q899 SA-050 • Intermediate

Find the highest-paid employee in each department (with ties).

⚡ **Quick answer**

`groupingBy(dept, collectingAndThen(maxBy(comparingInt(salary)), Optional::orElseThrow))` returns one employee per department. If two people share the top salary, `maxBy` silently picks one; to return **all** top earners, group to lists and filter each list by its maximum.

```java
Map<String, String> topEarner = employees.stream()
        .collect(Collectors.groupingBy(Employee::dept, TreeMap::new,
                Collectors.collectingAndThen(
                        Collectors.maxBy(Comparator.comparingInt(Employee::salary)),
                        e -> e.orElseThrow().name())));

Map<String, List<String>> allTopEarners = employees.stream()
        .collect(Collectors.groupingBy(Employee::dept, TreeMap::new, Collectors.toList()))
        .entrySet().stream()
        .collect(Collectors.toMap(Map.Entry::getKey, e -> {
            int max = e.getValue().stream().mapToInt(Employee::salary).max().orElseThrow();
            return e.getValue().stream()
                    .filter(x -> x.salary() == max).map(Employee::name).toList();
        }, (a, b) -> a, TreeMap::new));

System.out.println(topEarner);
System.out.println(allTopEarners);
```


© TechTribers. For personal use only. Do not redistribute.

▶ Output

```
{Engineering=Asha, Finance=Sneha, HR=Neha, Sales=Karan}
{Engineering=[Asha, Vikram], Finance=[Sneha], HR=[Neha], Sales=[Karan]}
```

⚠️ Interview trap

Asha and Vikram both earn 185,000 in Engineering. The single-winner version returns only one of them and hides the tie; the list version shows both.

Q900 SA-051

• Intermediate

Find the second-highest salary and every employee who earns it.

⚡ Quick answer

Compute it on **distinct salary values**, not on employees: `map(salary).distinct().sorted(reverseOrder()).skip(1).findFirst()` . Then filter employees by that value. Skipping one **employee** after sorting returns the top salary again whenever the top is shared.

Java

```
Optional<Integer> secondSalary = employees.stream()
        .map(Employee::salary)
        .distinct()
        .sorted(Comparator.reverseOrder())
        .skip(1)
        .findFirst();

List<String> earners = secondSalary
        .map(s -> employees.stream().filter(e -> e.salary() == s).map(Employee::name).toList())
        .orElse(List.of());

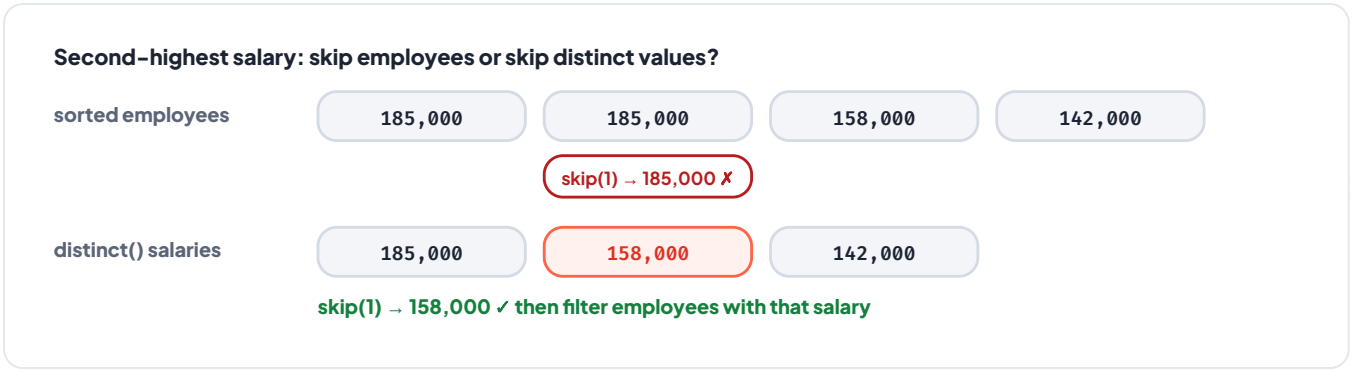
String naive = employees.stream()
        .sorted(Comparator.comparingInt(Employee::salary).reversed())
        .skip(1).findFirst().map(e -> e.name() + " " + e.salary()).orElse("-");

System.out.println(secondSalary.orElseThrow() + " " + earners);
System.out.println("naive answer: " + naive);
```

▶ Output

```
158000 [Sneha]
naive answer: Vikram 185000
```


© TechTribers. For personal use only. Do not redistribute.



Q901 SA-052

• Intermediate

Find employees who earn more than their department's average salary.

⚡ **Quick answer**

Two steps: compute the average per department **once** into a map, then filter employees against their own department's average. Computing the department average inside the filter for every employee is O(n²) and the most common mistake in this question.

Java

```
Map<String, Double> deptAvg = employees.stream()
        .collect(Collectors.groupingBy(Employee::dept, Collectors.averagingInt(Employee::salary)));

List<String> aboveAverage = employees.stream()
        .filter(e -> e.salary() > deptAvg.get(e.dept()))
        .sorted(Comparator.comparing(Employee::dept).thenComparing(Employee::name))
        .map(e -> e.dept() + ": " + e.name() + " " + e.salary()
                + " (avg " + Math.round(deptAvg.get(e.dept())) + ")")
        .toList();

aboveAverage.forEach(System.out::println);
```

▶ **Output**

```
Engineering: Asha 185000 (avg 170667)
Engineering: Vikram 185000 (avg 170667)
Finance: Sneha 158000 (avg 144500)
HR: Neha 76000 (avg 72500)
Sales: Karan 112000 (avg 104667)
```

💡 **Pro tip**

In SQL this is a window function (`salary > AVG(salary) OVER (PARTITION BY dept)`); for data already in a database, push it there instead of loading every row into Java.

© TechTribers. For personal use only. Do not redistribute.

Q902 SA-053

• Basic

Compute the total salary bill per city and find the costliest city.

⚡ Quick answer

`groupingBy(city, summingLong(salary))` (use `long` sums for money totals to avoid overflow with many employees), then `max(comparingByValue())` on the entries.

Java

```
Map<String, Long> billByCity = employees.stream()
        .collect(Collectors.groupingBy(Employee::city, TreeMap::new,
                Collectors.summingLong(Employee::salary)));

String costliest = billByCity.entrySet().stream()
        .max(Map.Entry.comparingByValue())
        .map(Map.Entry::getKey)
        .orElseThrow();

System.out.println(billByCity);
System.out.println("costliest city: " + costliest);
```

▶ Output

```
{Bengaluru=211000, Delhi=180000, Mumbai=229000, Pune=640000}
costliest city: Pune
```

Q903 SA-054

• Basic

List employees who joined after 2018, sorted by joining year and then name.

⚡ Quick answer

`filter(e -> e.joined() > 2018)`, then sort with `comparingInt(joined).thenComparing(name)`. Keep date logic in the predicate; in real code use `LocalDate` and `isAfter` rather than raw years.

Java

```
List<String> recentJoiners = employees.stream()
        .filter(e -> e.joined() > 2018)
        .sorted(Comparator.comparingInt(Employee::joined).thenComparing(Employee::name))
        .map(e -> e.joined() + " " + e.name() + " (" + e.dept() + ")")
        .toList();

recentJoiners.forEach(System.out::println);
```

▶ Output

```
2019 Asha (Engineering)
2021 Ravi (Engineering)
2022 Rahul (Sales)
```


© TechTribers. For personal use only. Do not redistribute.

```
2023 Neha (HR)
2024 Arjun (HR)
```

Q904 SA-055

• Basic

Find the oldest and youngest employee, and the average age by gender.

⚡ Quick answer

`max` / `min` with `comparingInt(age)` for the extremes; `groupingBy(gender, averagingInt(age))` for averages. `teeing` can compute oldest and youngest in one pass if the dataset is large.

Java

```java
Employee oldest = employees.stream().max(Comparator.comparingInt(Employee::age)).orElseThrow();
Employee youngest = employees.stream().min(Comparator.comparingInt(Employee::age)).orElseThrow();

Map<String, Double> avgAgeByGender = employees.stream()
        .collect(Collectors.groupingBy(Employee::gender, TreeMap::new,
                Collectors.averagingInt(Employee::age)));

System.out.println("oldest: " + oldest.name() + " " + oldest.age()
        + ", youngest: " + youngest.name() + " " + youngest.age());
System.out.println(avgAgeByGender);
```

▶ Output

```
oldest: Sneha 45, youngest: Arjun 24
{F=34.0, M=32.0}
```

Q905 SA-056

• Intermediate

Give a 10% raise to one department without mutating the original list.

⚡ Quick answer

Records are immutable, so `map` each employee to a **new** record with the updated salary when it matches, and leave others unchanged. Don't mutate elements from inside `forEach` or `map` : side effects in stream lambdas break with parallel streams and surprise readers.

Java

```java
List<Employee> afterRaise = employees.stream()
        .map(e -> e.dept().equals("Sales")
                ? new Employee(e.id(), e.name(), e.dept(), (int) Math.round(e.salary() * 1.10),
                        e.age(), e.gender(), e.city(), e.joined())
                : e)
        .toList();
```


© TechTribers. For personal use only. Do not redistribute.

```
afterRaise.stream()
        .filter(e -> e.dept().equals("Sales"))
        .forEach(e -> System.out.println(e.name() + " " + e.salary()));
System.out.println("original unchanged: " + employees.get(2).salary());
```

▶ Output

```
Meera 107800
Karan 123200
Rahul 114400
original unchanged: 98000
```

💡 Pro tip

A `withSalary(int)` method on the record keeps call sites readable; Java's planned derived record creation (`e with { salary = ... }`, JEP 468) is still a preview feature.

⚖️ Q906 SA-057 • Basic

Partition employees into those earning above and below 1,00,000, listing names.

⚡ Quick answer

`partitioningBy(e -> e.salary() > 100000, mapping(Employee::name, toList()))`. Both keys are always present, so callers can read `get(true)` and `get(false)` without null checks.

Java

```
Map<Boolean, List<String>> bySalaryBand = employees.stream()
        .sorted(Comparator.comparing(Employee::name))
        .collect(Collectors.partitioningBy(e -> e.salary() > 100_000,
                Collectors.mapping(Employee::name, Collectors.toList())));

System.out.println("above 1L: " + bySalaryBand.get(true));
System.out.println("at or below 1L: " + bySalaryBand.get(false));
```

▶ Output

```
above 1L: [Asha, Divya, Karan, Rahul, Ravi, Sneha, Vikram]
at or below 1L: [Arjun, Meera, Neha]
```


© TechTribers. For personal use only. Do not redistribute.

Q907 SA-058

• Intermediate

Produce salary statistics per department in one pass.

⚡ **Quick answer**

`groupingBy(dept, summarizingInt(salary))` gives an `IntSummaryStatistics` per department, with count, sum, min, max and average, computed in a single pass over the data.

Java

```
Map<String, IntSummaryStatistics> stats = employees.stream()
        .collect(Collectors.groupingBy(Employee::dept, TreeMap::new,
                Collectors.summarizingInt(Employee::salary)));

System.out.printf("%-12s %5s %9s %9s %11s%n", "dept", "count", "min", "max", "avg");
stats.forEach((dept, s) -> System.out.printf("%-12s %5d %9d %9d %11.1f%n",
        dept, s.getCount(), s.getMin(), s.getMax(), s.getAverage()));
```

▶ **Output**

```
dept         count       min       max         avg
Engineering      3    142000    185000    170666.7
Finance          2    131000    158000    144500.0
HR               2     69000     76000     72500.0
Sales            3     98000    112000    104666.7
```


© TechTribers. For personal use only. Do not redistribute.

Section 06

Real-World Data Pipelines

The stream problems that show up in production code: parsing and validating raw lines, in-memory joins and anti-joins, monthly aggregation, pivot tables, keeping the latest event per ID, histograms, reconciling two datasets and flattening recursive trees.

Questions 908–916 · 9 questions



Q908 SA-059

• Intermediate

Parse raw text lines into records, skipping the header and rejecting malformed rows.

⚡ Quick answer

`skip(1)` drops the header; a parse method returning `Optional` turns bad rows into empties, and `flatMap(Optional::stream)` keeps only valid records. Count or log the rejects instead of silently dropping them; in real imports, the number of rejected rows is a metric somebody needs.

Real CSV needs a real parser (quoted fields, embedded commas; see Q677). The stream structure stays the same: lines → parse → `Optional` → keep valid, report invalid.

Java

```
record Txn(String id, String account, BigDecimal amount, LocalDate date) { }

static Optional<Txn> parse(String line) {
    String[] f = line.split(",", -1);
    if (f.length != 4) return Optional.empty();
    try {
        return Optional.of(new Txn(f[0].strip(), f[1].strip(),
                new BigDecimal(f[2].strip()), LocalDate.parse(f[3].strip())));
    } catch (NumberFormatException | DateTimeParseException e) {
        return Optional.empty();
    }
}

List<String> lines = List.of(
        "id,account,amount,date",
        "T1,ACC-9,1500.50,2026-09-01",
        "T2,ACC-4,abc,2026-09-02",
        "T3,ACC-9,-200.00,2026-09-03",
        "T4,ACC-7,99.99",
        "T5,ACC-4,750.00,2026-10-01");

Map<Boolean, List<String>> byValidity = lines.stream().skip(1)
        .collect(Collectors.partitioningBy(l -> parse(l).isPresent()));
```


© TechTribers. For personal use only. Do not redistribute.

```
List<Txn> valid = byValidity.get(true).stream().map(Main::parse).flatMap(Optional::stream).toList();

valid.forEach(System.out::println);
System.out.println("rejected: " + byValidity.get(false));
```

▶ Output

```
Txn[id=T1, account=ACC-9, amount=1500.50, date=2026-09-01]
Txn[id=T3, account=ACC-9, amount=-200.00, date=2026-09-03]
Txn[id=T5, account=ACC-4, amount=750.00, date=2026-10-01]
rejected: [T2,ACC-4,abc,2026-09-02, T4,ACC-7,99.99]
```

Q909 SA-060 • Intermediate

Join orders with customers in memory, like a SQL inner join.

⚡ Quick answer

Index the smaller side by key **once** (`toMap(Customer::id, identity())`), then stream the larger side and look each key up: an O(n + m) hash join. Streaming customers inside a `filter` for every order is a nested-loop join, O(n × m).

Java

```
record Customer(int id, String name, String tier) { }
record Order(String id, int customerId, int amount) { }
record OrderView(String orderId, String customer, String tier, int amount) { }

List<Customer> customers = List.of(new Customer(1, "Asha", "GOLD"),
        new Customer(2, "Ravi", "SILVER"), new Customer(3, "Meera", "GOLD"));
List<Order> orders = List.of(new Order("O1", 1, 1200), new Order("O2", 3, 450),
        new Order("O3", 9, 300), new Order("O4", 1, 80));

Map<Integer, Customer> byId = customers.stream()
        .collect(Collectors.toMap(Customer::id, Function.identity()));

List<OrderView> joined = orders.stream()
        .filter(o -> byId.containsKey(o.customerId()))
        .map(o -> {
            Customer c = byId.get(o.customerId());
            return new OrderView(o.id(), c.name(), c.tier(), o.amount());
        })
        .toList();

joined.forEach(System.out::println);
```

▶ Output

```
OrderView[orderId=O1, customer=Asha, tier=GOLD, amount=1200]
OrderView[orderId=O2, customer=Meera, tier=GOLD, amount=450]
OrderView[orderId=O4, customer=Asha, tier=GOLD, amount=80]
```


© TechTribers. For personal use only. Do not redistribute.

💡 **Pro tip**

Order `O3` references customer 9, which doesn't exist, so an inner join drops it. For a left join, keep it with a placeholder (`byId.getOrDefault(id, UNKNOWN)`), and report orphan references either way.

🚫 Q910 SA-061

• Basic

Find customers who have no orders (an anti-join) with streams.

⚡ **Quick answer**

Collect the referenced keys from one side into a `Set` , then filter the other side with `not(set::contains)` . It's the stream equivalent of `NOT EXISTS` / `LEFT JOIN ... WHERE right.id IS NULL` , in O(n + m).

Java

```
record Customer(int id, String name) { }
record Order(String id, int customerId) { }

List<Customer> customers = List.of(new Customer(1, "Asha"), new Customer(2, "Ravi"),
        new Customer(3, "Meera"), new Customer(4, "Karan"));
List<Order> orders = List.of(new Order("O1", 1), new Order("O2", 3), new Order("O3", 1));

Set<Integer> withOrders = orders.stream().map(Order::customerId).collect(Collectors.toSet());

List<String> inactive = customers.stream()
        .filter(c -> !withOrders.contains(c.id()))
        .map(Customer::name)
        .toList();

System.out.println(inactive);
```

▶ **Output**

```
[Ravi, Karan]
```

🗓️ Q911 SA-062

• Intermediate

Aggregate transactions by month (`YearMonth`) and compute monthly totals.

⚡ **Quick answer**

Group by `YearMonth.from(date)` into a `TreeMap` (chronological order) with a `BigDecimal` sum via `reducing(BigDecimal.ZERO, Txn::amount, BigDecimal::add)` . Use `BigDecimal` for money; `summingDouble` accumulates floating-point error.

© TechTribers. For personal use only. Do not redistribute.

```java
record Txn(LocalDate date, BigDecimal amount) { }

List<Txn> txns = List.of(
        new Txn(LocalDate.of(2026, 8, 3), new BigDecimal("1200.10")),
        new Txn(LocalDate.of(2026, 8, 28), new BigDecimal("349.90")),
        new Txn(LocalDate.of(2026, 9, 1), new BigDecimal("0.10")),
        new Txn(LocalDate.of(2026, 9, 1), new BigDecimal("0.20")),
        new Txn(LocalDate.of(2026, 10, 15), new BigDecimal("75.00")));

Map<YearMonth, BigDecimal> monthly = txns.stream()
        .collect(Collectors.groupingBy(t -> YearMonth.from(t.date()), TreeMap::new,
                Collectors.reducing(BigDecimal.ZERO, Txn::amount, BigDecimal::add)));

double viaDouble = txns.stream().filter(t -> t.date().getMonthValue() == 9)
        .mapToDouble(t -> t.amount().doubleValue()).sum();

System.out.println(monthly);
System.out.println("September as double: " + viaDouble);
```

▶ Output

```
{2026-08=1550.00, 2026-09=0.30, 2026-10=75.00}
September as double: 0.30000000000000004
```

Q912 SA-063 • Advanced

Build a pivot table (region by quarter) with zero-filled cells.

⚡ Quick answer

Group by the row key, then by the column key, summing values: `groupingBy(region, TreeMap::new, groupingBy(quarter, TreeMap::new, summingInt(amount)))`. `groupingBy` only creates cells that have data, so fill missing quarters with zero before rendering, or the report has gaps.

```java
record Sale(String region, int quarter, int amount) { }

List<Sale> sales = List.of(new Sale("North", 1, 120), new Sale("North", 2, 90),
        new Sale("South", 1, 70), new Sale("North", 1, 30), new Sale("West", 4, 200),
        new Sale("South", 3, 55));

Map<String, Map<Integer, Integer>> pivot = sales.stream()
        .collect(Collectors.groupingBy(Sale::region, TreeMap::new,
                Collectors.groupingBy(Sale::quarter, TreeMap::new,
                        Collectors.summingInt(Sale::amount))));

System.out.printf("%-6s %5s %5s %5s %5s %6s%n", "", "Q1", "Q2", "Q3", "Q4", "total");
pivot.forEach((region, byQ) -> {
    String cells = IntStream.rangeClosed(1, 4)
```


© TechTribers. For personal use only. Do not redistribute.

```
            .mapToObj(q -> String.format("%5d", byQ.getOrDefault(q, 0)))
            .collect(Collectors.joining(" "));
    int total = byQ.values().stream().mapToInt(Integer::intValue).sum();
    System.out.printf("%-6s %s %6d%n", region, cells, total);
});
```

▶ Output

```
          Q1    Q2    Q3    Q4  total
North    150    90     0     0    240
South     70     0    55     0    125
West       0     0     0   200    200
```

Q913 SA-064 • Intermediate

Deduplicate events by ID, keeping only the latest version of each.

⚡ Quick answer

`toMap(Event::id, identity(), BinaryOperator.maxBy(comparing(Event::version)))` : on a key collision the merge function keeps whichever event has the higher version (or timestamp). This is the stream form of compacting a change log, as a Kafka compacted topic or a CDC consumer does.

Note `ORD-1` v2 arrives after v3 in the input; ordering by version instead of by arrival makes the result correct even with out-of-order delivery. Use the event's own version or sequence number, not the time you received it.

Java

```
record Event(String id, int version, String status, Instant at) { }

List<Event> changes = List.of(
        new Event("ORD-1", 1, "CREATED", Instant.parse("2026-10-01T10:00:00Z")),
        new Event("ORD-2", 1, "CREATED", Instant.parse("2026-10-01T10:05:00Z")),
        new Event("ORD-1", 3, "SHIPPED", Instant.parse("2026-10-02T09:00:00Z")),
        new Event("ORD-1", 2, "PAID", Instant.parse("2026-10-01T12:00:00Z")),
        new Event("ORD-2", 2, "CANCELLED", Instant.parse("2026-10-01T11:00:00Z")));

Map<String, Event> latest = changes.stream()
        .collect(Collectors.toMap(Event::id, Function.identity(),
                BinaryOperator.maxBy(Comparator.comparingInt(Event::version)),
                TreeMap::new));

latest.values().forEach(e -> System.out.println(e.id() + " v" + e.version() + " " + e.status()));
```

▶ Output

```
ORD-1 v3 SHIPPED
ORD-2 v2 CANCELLED
```


© TechTribers. For personal use only. Do not redistribute.

Q914 SA-065

• Intermediate

Bucket values into ranges (a histogram of salary bands).

⚡ Quick answer

Map each value to its bucket's lower bound with integer division (`salary / 50_000 * 50_000`), group into a `TreeMap` with `counting()` , and label each bucket for display. Pre-fill empty buckets if the chart must show them.

Java

```
List<Integer> salaries = List.of(185000, 142000, 98000, 112000, 76000, 185000,
        131000, 69000, 158000, 104000, 51000, 47000);
int width = 50_000;

Map<Integer, Long> histogram = salaries.stream()
        .collect(Collectors.groupingBy(s -> s / width * width, TreeMap::new,
                Collectors.counting()));

histogram.forEach((low, count) -> System.out.printf("%,7d - %,7d | %s %d%n",
        low, low + width - 1, "#".repeat(count.intValue()), count));
```

▶ Output

```
      0 -  49,999 | # 1
 50,000 -  99,999 | #### 4
100,000 - 149,999 | #### 4
150,000 - 199,999 | ### 3
```

Q915 SA-066

• Advanced

Reconcile two datasets by key: matched, changed, only in source, only in target.

⚡ Quick answer

Index both sides by key into maps, take the **union** of the key sets, and classify each key: present in both and equal, present in both but different, only in the source, only in the target. `groupingBy(classifier, TreeMap::new, toList())` produces the whole report in one pass over the union.

Java

```
record Balance(String account, int cents) { }

List<Balance> ledger = List.of(new Balance("A1", 1000), new Balance("A2", 2500),
        new Balance("A3", 700), new Balance("A5", 50));
List<Balance> bank = List.of(new Balance("A1", 1000), new Balance("A2", 2400),
        new Balance("A4", 900), new Balance("A5", 50));

Map<String, Integer> src = ledger.stream()
```


© TechTribers. For personal use only. Do not redistribute.

```
        .collect(Collectors.toMap(Balance::account, Balance::cents));
Map<String, Integer> dst = bank.stream()
        .collect(Collectors.toMap(Balance::account, Balance::cents));

Map<String, List<String>> report = Stream.concat(src.keySet().stream(), dst.keySet().stream())
        .distinct()
        .sorted()
        .collect(Collectors.groupingBy(k -> {
            if (!dst.containsKey(k)) return "only in ledger";
            if (!src.containsKey(k)) return "only in bank";
            return src.get(k).equals(dst.get(k)) ? "matched" : "amount differs";
        }, TreeMap::new, Collectors.toList()));

report.forEach((status, accounts) -> System.out.println(status + ": " + accounts));
```

▶ Output

```
amount differs: [A2]
matched: [A1, A5]
only in bank: [A4]
only in ledger: [A3]
```

💡 Pro tip

Compare boxed `Integer` s with `equals` , never `==` : outside the cache range of -128 to 127, two equal amounts are usually different objects and `==` reports a false mismatch.

🌳 Q916 SA-067 • Advanced

Flatten a recursive tree of categories into a list with depth and path.

⚡ Quick answer

Write a recursive method that returns a `Stream` : the node itself followed by `children.stream().flatMap(child -> flatten(child, depth + 1))` . Because streams are lazy, the tree is walked depth-first as elements are pulled; the result can be filtered, limited or collected like any other stream.

Java

```
record Category(String name, List<Category> children) {
    Category(String name, Category... kids) { this(name, List.of(kids)); }
}
record Flat(int depth, String path) { }

static Stream<Flat> flatten(Category c, int depth, String parent) {
    String path = parent.isEmpty() ? c.name() : parent + " > " + c.name();
    return Stream.concat(Stream.of(new Flat(depth, path)),
            c.children().stream().flatMap(k -> flatten(k, depth + 1, path)));
}

Category root = new Category("Electronics",
```


© TechTribers. For personal use only. Do not redistribute.

```
        new Category("Phones", new Category("Android"), new Category("iOS")),
        new Category("Laptops", new Category("Gaming"), new Category("Ultrabooks")),
        new Category("Audio"));

flatten(root, 0, "").forEach(f -> System.out.println("  ".repeat(f.depth()) + f.path()));
long leaves = flatten(root, 0, "").filter(f -> f.depth() == 2).count();
System.out.println("depth-2 categories: " + leaves);
```

▶ Output

```
Electronics
  Electronics > Phones
    Electronics > Phones > Android
    Electronics > Phones > iOS
  Electronics > Laptops
    Electronics > Laptops > Gaming
    Electronics > Laptops > Ultrabooks
  Electronics > Audio
depth-2 categories: 4
```

⚠️ **Interview trap**

Very deep trees (thousands of levels) can overflow the stack, because each level adds nested `flatMap` and `concat` frames. For unbounded depth, iterate with an explicit `Deque`.

Section 07

Advanced: Generation, Laziness and Modern Stream Features

Fibonacci and primes with Stream.iterate, safe infinite streams, BigInteger factorials, lazy evaluation order and short-circuiting, why streams can't be reused, Gatherers for running sums and windows, Optional::stream to drop empties, and run-length encoding with regex streams.

Questions 917–924 · 8 questions

Q917 SA-068

• Intermediate

Generate the first N Fibonacci numbers and the first N primes with streams.

⚡ **Quick answer**

Fibonacci needs the previous two values as state, so iterate over **pairs**: `Stream.iterate(new long[]{0, 1}, f -> new long[]{f[1], f[0] + f[1]})` and map to the first element. Primes: `IntStream.iterate(2, i -> i + 1).filter(isPrime).limit(n)`, where `isPrime` checks divisors up to the square root.

© TechTribers. For personal use only. Do not redistribute.

```java
static boolean isPrime(int n) {
    return n > 1 && IntStream.rangeClosed(2, (int) Math.sqrt(n)).noneMatch(d -> n % d == 0);
}

List<Long> fib = Stream.iterate(new long[]{0, 1}, f -> new long[]{f[1], f[0] + f[1]})
        .limit(12)
        .map(f -> f[0])
        .toList();

List<Integer> primes = IntStream.iterate(2, i -> i + 1)
        .filter(Main::isPrime)
        .limit(10)
        .boxed()
        .toList();

System.out.println(fib);
System.out.println(primes);
```

▶ Output

```
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
[2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
```

Q918 SA-069 • Intermediate

Create infinite streams safely with `iterate`, `generate` and seeded random numbers.

⚡ Quick answer

Infinite sources need a **short-circuiting** operation (`limit`, `takeWhile`, `findFirst`, `anyMatch`). Java 9's three-argument `Stream.iterate(seed, hasNext, next)` works like a `for` loop and terminates by itself. `Stream.generate(supplier)` is unordered and has no state of its own. Seed `Random` in tests so output is reproducible.

```java
List<Integer> powersOf2 = Stream.iterate(1, x -> x <= 1_000, x -> x * 2).toList();

List<String> ids = Stream.generate(() -> "id").limit(3).toList();

Random seeded = new Random(42);
List<Integer> dice = seeded.ints(6, 1, 7).boxed().toList();

int firstOver = IntStream.iterate(1, x -> x * 3).filter(x -> x > 500).findFirst().getAsInt();

System.out.println(powersOf2);
System.out.println(ids + " " + dice + " " + firstOver);
```


© TechTribers. For personal use only. Do not redistribute.

▶ Output

```
[1, 2, 4, 8, 16, 32, 64, 128, 256, 512]
[id, id, id] [3, 4, 1, 3, 1, 2] 729
```

⚠️ Interview trap

`IntStream.iterate(1, x -> x * 3).filter(x -> x < 0).findFirst()` looks infinite but stops once `int` overflows into a negative number, which is a wrong answer rather than a hang. Bound infinite streams by something meaningful.

Q919 SA-070 • Intermediate

Compute a large factorial and other aggregates with primitive streams and `BigInteger`.

⚡ Quick answer

Use `IntStream.rangeClosed(1, n)` for ranges and `LongStream` for products up to 20!; beyond that, `mapToObj(BigInteger::valueOf).reduce(BigInteger.ONE, BigInteger::multiply)`. Primitive streams avoid boxing and provide `sum`, `average`, `summaryStatistics` directly.

Java

```
int sum1to100 = IntStream.rangeClosed(1, 100).sum();
long fact20 = LongStream.rangeClosed(1, 20).reduce(1, (a, b) -> a * b);
long overflow21 = LongStream.rangeClosed(1, 21).reduce(1, (a, b) -> a * b);

BigInteger fact30 = IntStream.rangeClosed(1, 30)
        .mapToObj(BigInteger::valueOf)
        .reduce(BigInteger.ONE, BigInteger::multiply);

double avgOfSquares = IntStream.rangeClosed(1, 10).map(x -> x * x).average().orElse(0);

System.out.println(sum1to100 + " " + fact20);
System.out.println("21! as long (overflowed): " + overflow21);
System.out.println("30! = " + fact30);
System.out.println(avgOfSquares);
```

▶ Output

```
5050 2432902008176640000
21! as long (overflowed): -4249290049419214848
30! = 265252859812191058636308480000000
38.5
```


© TechTribers. For personal use only. Do not redistribute.

Q920 SA-071 • Advanced

What does this pipeline print? (lazy evaluation, operation order and short-circuiting)

⚡ Quick answer

Streams are **lazy** and process elements **one at a time through the whole pipeline** (vertically), not stage by stage. Nothing runs until a terminal operation. `findFirst` stops as soon as one element passes, so later elements are never mapped. `sorted` is a **stateful barrier**: it must see every element before passing any downstream.

Java

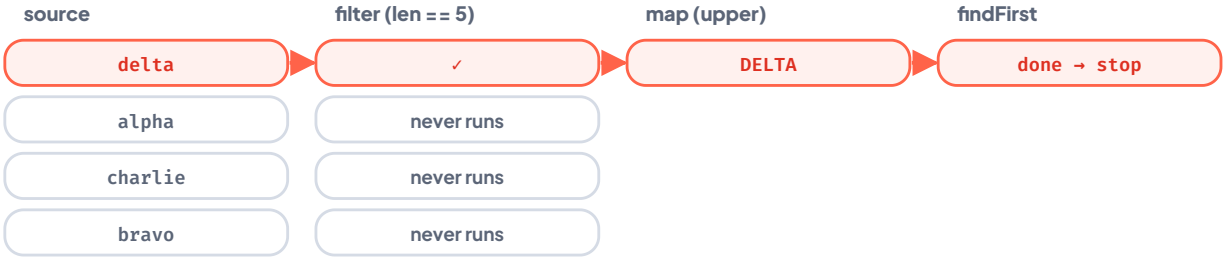
```
Stream<String> pipeline = Stream.of("delta", "alpha", "charlie", "bravo")
        .filter(s -> { System.out.println("filter " + s); return s.length() == 5; })
        .map(s -> { System.out.println("map    " + s); return s.toUpperCase(); });
System.out.println("-- nothing has run yet --");
System.out.println("result: " + pipeline.findFirst().orElseThrow());

System.out.println("-- with sorted() in the middle --");
String first = Stream.of("delta", "alpha", "charlie")
        .peek(s -> System.out.println("before sort " + s))
        .sorted()
        .peek(s -> System.out.println("after sort  " + s))
        .findFirst().orElseThrow();
System.out.println("result: " + first);
```

▶ Output

```
-- nothing has run yet --
filter delta
map    delta
result: DELTA
-- with sorted() in the middle --
before sort delta
before sort alpha
before sort charlie
after sort  alpha
result: alpha
```

Streams run element by element through the pipeline (vertical), not stage by stage



sorted() is a barrier: it must receive every element before emitting the first one.

© TechTribers. For personal use only. Do not redistribute.

⚠️ **Interview trap**

Using `peek` for real side effects. Since Java 9, a stream may even skip pipeline stages entirely when the result is known from the source (for example `count()` on a sized list without `filter`), so `peek` doesn't run at all.

Q921 SA-072 • Advanced

Use Stream Gatherers (Java 24) for a running sum, fixed and sliding windows.

⚡ **Quick answer**

`Stream.gather(Gatherer)` adds custom intermediate operations. Built-ins in `java.util.stream.Gatherers`: `scan` (running accumulation, emitting every intermediate value), `fold` (a single result), `windowFixed(n)` (non-overlapping chunks), `windowSliding(n)` (overlapping windows) and `mapConcurrent`. They replace the awkward index-based workarounds from Q869.

Gatherers were previewed in Java 22 and 23 and became final in **Java 24** (JEP 485); this snippet runs on JDK 25 without preview flags. Q404 covers how to write your own `Gatherer`.

Java

```
List<Integer> daily = List.of(4, 8, 6, 3, 9, 7, 5);

List<Integer> running = daily.stream()
        .gather(Gatherers.scan(() -> 0, Integer::sum))
        .toList();

List<List<Integer>> weeks = daily.stream()
        .gather(Gatherers.windowFixed(3))
        .toList();

List<Double> movingAvg = daily.stream()
        .gather(Gatherers.windowSliding(3))
        .map(w -> w.stream().mapToInt(Integer::intValue).average().orElse(0))
        .toList();

System.out.println("running sum: " + running);
System.out.println("fixed:       " + weeks);
System.out.println("moving avg:  " + movingAvg);
```

▶ **Output**

```
running sum: [4, 12, 18, 21, 30, 37, 42]
fixed:       [[4, 8, 6], [3, 9, 7], [5]]
moving avg:  [6.0, 5.666666666666667, 6.0, 6.333333333333333, 7.0]
```


© TechTribers. For personal use only. Do not redistribute.

Gatherers (Java 24) on [4, 8, 6, 3, 9, 7, 5]

input	4 | 8 | 6 | 3 | 9 | 7 | 5
`scan(0, +)`	`4, 12, 18, 21, 30, 37, 42`
`windowFixed(3)`	`[4,8,6] [3,9,7] [5]`
`windowSliding(3)`	`[4,8,6] [8,6,3] [6,3,9] [3,9,7] [9,7,5]`
`fold(0, +)`	`42 (single result)`

💡 **Pro tip**

`windowFixed` emits a shorter **last** window when the size doesn't divide evenly (`[5]` above), which is usually what you want for batching.

♻️ **Q922** SA-073 • Intermediate

Why can a stream be consumed only once, and how do you reuse a pipeline?

⚡ **Quick answer**

A `Stream` is a one-shot view over a source, not a data structure. After a terminal operation it's **consumed**, and any further operation throws `IllegalStateException: stream has already been operated upon or closed`. To reuse a pipeline, keep a `Supplier<Stream<T>>` (or a method) that builds a fresh stream each time, or collect once into a list.

Java

```
List<String> cities = List.of("Pune", "Patna", "Delhi", "Panaji");

Stream<String> pStream = cities.stream().filter(c -> c.startsWith("P"));
System.out.println(pStream.count());
try {
    pStream.toList();
} catch (IllegalStateException e) {
    System.out.println(e.getMessage());
}

Supplier<Stream<String>> pCities = () -> cities.stream().filter(c -> c.startsWith("P"));
System.out.println(pCities.get().count() + " " + pCities.get().toList());
```

▶ **Output**

```
3
stream has already been operated upon or closed
3 [Pune, Patna, Panaji]
```


© TechTribers. For personal use only. Do not redistribute.

Q923 SA-074

• Intermediate

Drop empty `Optional`s from a stream and flatten the present values.

⚡ Quick answer

Since Java 9, `Optional::stream` turns an `Optional` into a stream of zero or one element, so `flatMap(Optional::stream)` keeps only present values. It replaces `filter(Optional::isPresent).map(Optional::get)`.

Java

```
static Optional<Integer> parsePort(String s) {
    try {
        int p = Integer.parseInt(s.strip());
        return p > 0 && p < 65_536 ? Optional.of(p) : Optional.empty();
    } catch (NumberFormatException e) {
        return Optional.empty();
    }
}

List<String> raw = List.of("8080", "abc", " 443 ", "70000", "5432");

List<Integer> ports = raw.stream()
        .map(Main::parsePort)
        .flatMap(Optional::stream)
        .toList();

System.out.println(ports);
```

▶ Output

```
[8080, 443, 5432]
```

Q924 SA-075

• Intermediate

List every Monday between two dates, and count working days, with `LocalDate.datesUntil`.

⚡ Quick answer

`LocalDate.datesUntil(end)` (Java 9) returns a `Stream<LocalDate>` of each day from the start (inclusive) to the end (exclusive); the overload `datesUntil(end, Period)` steps by any period. Filter by `getDayOfWeek()` for weekdays, and subtract a holiday set for working days.

Java

```
LocalDate from = LocalDate.of(2026, 10, 1);
LocalDate to = LocalDate.of(2026, 11, 1);
Set<LocalDate> holidays = Set.of(LocalDate.of(2026, 10, 2), LocalDate.of(2026, 10, 20));
```


© TechTribers. For personal use only. Do not redistribute.

```
List<LocalDate> mondays = from.datesUntil(to)
        .filter(d -> d.getDayOfWeek() == DayOfWeek.MONDAY)
        .toList();

long workingDays = from.datesUntil(to)
        .filter(d -> d.getDayOfWeek().getValue() <= 5)
        .filter(d -> !holidays.contains(d))
        .count();

List<LocalDate> monthStarts = LocalDate.of(2026, 1, 1)
        .datesUntil(LocalDate.of(2026, 7, 1), Period.ofMonths(1))
        .toList();

System.out.println(mondays);
System.out.println("working days in October 2026: " + workingDays);
System.out.println(monthStarts);
```

▶ Output

```
[2026-10-05, 2026-10-12, 2026-10-19, 2026-10-26]
working days in October 2026: 20
[2026-01-01, 2026-02-01, 2026-03-01, 2026-04-01, 2026-05-01, 2026-06-01]
```

💡 **Pro tip**

The end date is exclusive, like `IntStream.range`. Use `to.plusDays(1)` when the business rule says "up to and including".

© TechTribers. For personal use only. Do not redistribute.

Series progress

924 unique questions so far



Every question in the TechTribers Java series has one number, with duplicates removed across all parts. Part 19 starts at Q925.

Part	Questions	Count
Part 1	Q1–Q74	74
Part 2	Q75–Q121	47
Part 3	Q122–Q170	49
Part 4	Q171–Q204	34
Part 5	Q205–Q254	50
Part 6	Q255–Q304	50
Part 7	Q305–Q339	35
Part 8	Q340–Q372	33
Part 9	Q373–Q397	25
Part 10	Q398–Q427	30
Part 11	Q428–Q479	52
Part 12	Q480–Q529	50
Part 13	Q530–Q579	50
Part 14	Q580–Q679	100
Part 15	Q680–Q739	60
Part 16	Q740–Q799	60
Part 17	Q800–Q849	50
Part 18	Q850–Q924	75

More interview prep, carousels and roadmaps at **techtriber.com** and on Instagram **@techtribers**.

© TechTribers. For personal use only. Do not redistribute.